

ECE 218 — Fall 2026 — Solutions

Sample Exam Questions (Sample1.pdf) and Practice 1 — Simple Smart Home (Practice1.pdf). Every code sample was compiled with g++ and clang++ (-Wall -Wextra), run, and checked with valgrind. Every sorting trace and count was checked by running the instructor's algorithms.

Contents

1. Sample1 — General Questions G1–G20
2. Sample1 — Long Questions L1 (Mergesort), L2 (Selection sort trace), L3 (Customer class), L4 (Address design)
3. Practice 1 — Simple Smart Home (full code, data file, output)

Part 1 — Sample1: General Questions

Compiling, memory and pointers

G1. What are the stages in compiling a C++ program into an executable

Answer

Four stages, always in this order. `g++ sort.cpp support.cpp -o sort` runs all four automatically.

1. **Preprocessing** (`g++ -E sort.cpp -o sort.ii`). The preprocessor handles every line that starts with `#`:
 - `#include` pastes in the header file (e.g. `hw.h`, `support.h`).
 - `#define` macros are expanded.
 - `#ifdef` / `#ifndef` / `#endif` keep or drop code. An example is the include guard `#ifndef HW_H ... #endif`, which makes sure a header is pasted only once.
 - Comments are removed.

Instructor example: in `hw.h`, `#ifdef DEBUG` sets `PDEBUG` to 1 (otherwise 0), so the `DPRINT` debug macro prints only when compiled with `-DDEBUG`. **Output:** plain C++ text with no `#` directives (`.ii`).

2. **Compilation** (`g++ -S sort.cpp` produces `sort.s`). The compiler checks syntax and types, so syntax and type errors are reported here. It optimizes and translates the C++ into **assembly language** for the target CPU.
3. **Assembly** (`g++ -c sort.cpp` produces `sort.o`; `-c` means stop after this stage). The assembler turns the assembly into machine code, which is an **object file**. It cannot run yet, because calls to functions defined in other files are still unresolved symbols. For example, `sort.cpp` calls `getCPUtime()`, which is defined in `support.cpp`.
4. **Linking** (`g++ sort.o support.o -o sort`). The linker combines all the `.o` files and the C++ standard library, connects every call to its definition (resolves the symbols), and writes the **executable** (`a.out` if no `-o` is given). A missing definition gives `undefined reference to ...`. That is a **link** error, not a compile error.

Execution: `./sort 5 < d5.txt`. The OS loader places the program in memory (code R/O, globals R/W, heap, stack), and the start-up code then calls `main()`.

Why separate .o files help: after a change, only the changed `.cpp` is recompiled, and then everything is relinked.

Worked steps / details



still unresolved)



Build commands (run on the instructor's own sort.cpp and support.cpp with g++ 13, -Wall -Wextra):

```
g++ -Wall -Wextra -c support.cpp    # support.cpp -> support.o
g++ -Wall -Wextra -c sort.cpp       # sort.cpp -> sort.o
g++ sort.o support.o -o sort        # link -> executable
./sort 5 < d5.txt                  # run it
g++ -save-temps -c sort.cpp         # keeps sort.ii, sort.s, sort.o (one file per stage)
```

How the instructor's DPRINT macro works (hw.h, lightly simplified; compiles cleanly):

```
#ifdef DEBUG
#define PDEBUG 1
#else
#define PDEBUG 0
#endif
#define DPRINT(fmt, ...) do { if (PDEBUG) { \
    char str[100]; snprintf(str, 100, fmt, ##__VA_ARGS__); \
    std::cerr << "DEBUG-->" << __FILE__ << ":" << __LINE__ << ":" \
        << __func__ << "(): " << str; \
    } } while (0)
```

- g++ -DDEBUG works like writing #define DEBUG 1 at the top of the file.
- After stage 1 (g++ -E hello2.cpp), the line DPRINT("i=%d\n",i); becomes do { if (0) { ... "hello2.cpp" << ":" << 25 ... } } while (0);. With -DDEBUG it is the same text with if (1). The preprocessor has also filled in __FILE__ and __LINE__.
- In stage 2 the compiler drops the if (0) block as dead code. With -DDEBUG, the run prints DEBUG-->hello2.cpp:25:main():i=0.

Which stage each error comes from (all verified):

- **Stage 1:** a missing header gives fatal error: hw.h: No such file or directory.
- **Stage 2:** syntax and type errors, such as a missing ;, an undeclared name or the wrong type.
- **Stage 4:** undefined reference or multiple definition. Linking sort.o alone gives undefined reference to 'getCPUtime()'. Linking main.o without student.o gives undefined reference to 'Student::Student()'.
Linking student.o alone gives multiple definition of 'Student::print()'.
- **Run time:** segfaults, stack smashing and memory leaks.

Tie-ins:

- The class declaration goes in student.h and the definitions (Student::print) in student.cpp. Each .cpp is compiled on its own and the linker joins them: g++ -c student.cpp, then g++ -c main.cpp, then g++ main.o student.o -o student.
- Template code must be in the .h file, because the compiler has to see the source in stage 2 to generate bubbleT<int>.

الشرح بالعربي خطوة بخطوة

الفكرة الأساسية: المعالج لا يفهم نص لغة سي بلس بلس مباشرة، لذلك يمر ملفك بأربع محطات متتالية حتى يصبح برنامجاً يمكن تشغيله. احفظها بالترتيب: معالجة مسبقة، ثم ترجمة، ثم تجميع، ثم ربط.

(١) المعالجة المسبقة: المعالج المسبق يقرأ فقط الأسطر التي تبدأ بالعلامة

#

فينسخ محتوى ملف الترويسة مكان سطر التضمنين، ويستبدل كل ماكرو بتعريفه، ويحذف التعليقات، ويقرر أي الأجزاء تبقى وأياها تُحذف حسب الشروط. مثال الأستاذ: ماكرو الطباعة للتصحيح في الملف

hw.h

لا يطبع إلا إذا ترجمنا مع الخيار

-DDEBUG

لأن المعالج المسبق يضع مكان مفتاح التصحيح واحداً مع هذا الخيار، وصفراً بدونه.

الناتج ملف نصي كبير ما زال بلغة سي بلس بلس، لكن بدون أي سطر يبدأ بتلك العلامة.

(٢) الترجمة: المترجم يفحص القواعد والأنواع، وهنا تظهر الأخطاء النحوية. ثم يحول الشيفرة إلى لغة التجميع الخاصة بالمعالج.

(٣) التجميع: المجمع يحول لغة التجميع إلى لغة الآلة، والناتج ملف كائني. هذا الملف لا يعمل وحده، لأن فيه استدعاءات لدوال موجودة في ملفات أخرى لم تُربط بعد. مثلاً برنامج الفرز عند الأستاذ يستدعي دالة قياس الوقت، وتعريفها موجود في ملف الدعم.

(٤) الربط: الرابط يجمع كل الملفات الكائنية مع المكتبة القياسية، ويربط كل استدعاء بتعريف الدالة الحقيقي، والناتج هو الملف التنفيذي. إذا نسيت ملفاً كائنياً يظهر خطأ ربط وليس خطأ ترجمة، ورسالته:

undefined reference

بعد ذلك يأتي التنفيذ: نظام التشغيل يحمل البرنامج في الذاكرة (منطقة الشيفرة، المتغيرات العامة، الكومة، المكس)، ثم يبدأ التنفيذ من الدالة الرئيسية.

فائدة تقسيم المراحل: إذا عدلت ملفاً واحداً فإنك تعيد ترجمته وحده ثم تعيد الربط، بدل ترجمة المشروع كله.

Lecture link: 8/20-24 lecture: hw.h has an include guard (#ifndef HW_H_) and the DPRINT debug macro. #ifdef DEBUG sets PDEBUG to 1 or 0, and DPRINT(fmt, ...) expands to do { if (PDEBUG) {...} } while(0), so it prints only when compiled with -DDEBUG (shown with hello2.cpp). The instructor's multi-

file sort program is sort.cpp plus support.cpp/support.h, where getCPUtime() is defined in support.cpp and linked in. 9/15: template code goes in the .h because the compiler needs the source. 9/17: class declaration in student.h and definition in student.cpp (Student::print), compiled separately and linked with main.cpp.

G2. What is the difference between automatic allocation and dynamic allocation

Answer

	Automatic	Dynamic	Static (for completeness)
What	local variables, local arrays, parameters	memory requested at run time with <code>new / new[]</code> (C: <code>malloc</code>)	global variables, static variables
Where	stack , in the function's stack frame	heap	global / static data area
Created	automatically when the function is called	when <code>new</code> executes	when the program is loaded
Freed	automatically when the function returns (frame popped)	only when the programmer calls <code>delete / delete []</code>	when the program ends
Size	fixed at compile time	can be decided at run time (<code>new int[n]</code>)	fixed at compile time
Access	by name	only through a pointer	by name

- **Automatic:** fast, and it needs no cleanup. It only lasts as long as the function, and the stack size limits it. Huge local arrays cause a stack overflow, and writing past the end of a local array causes stack smashing.
- **Dynamic:** the size and lifetime are flexible, because the block survives the function that created it. The programmer is responsible for freeing it. Forgetting `delete` causes a **memory leak**, and using the memory after `delete` is a use-after-deallocation bug.

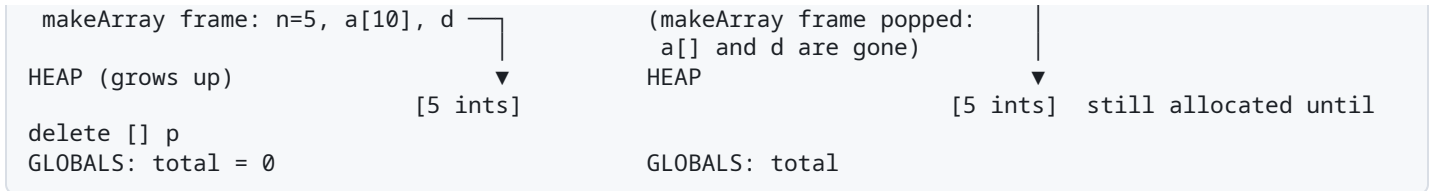
```
int total = 0; // static: global area, whole program
int *makeArray(int n) {
    int a[10]; // automatic: in makeArray's stack frame
    a[0] = n;
    int *d = new int[n]; // dynamic: n ints on the heap, n chosen at run time
    d[0] = a[0];
    return d; // OK: heap block outlives the function
} // a[] destroyed here (returning a would be a bug)
// caller: int *p = makeArray(n); ... delete [] p; p = nullptr;
```

This is the same pattern as the instructor's `loadData()` in sort.cpp: `int *temp = new int[n]; ... return temp;`, with the comment "allocates memory in heap, user must deallocate".

Worked steps / details

During `makeArray(5)`
 STACK (grows down)
 main frame: `n=5, p=?`

After it returns to main
 STACK
 main frame: `n=5, p` —————



Rules of thumb:

- **Automatic** = stack, and it is freed for you.
- **Dynamic** = heap, and YOU free it: `new` with `delete`, `new[]` with `delete []`.
- **Static** = global area, and it lives for the whole run.

Verified: the program above, with a `main` that calls `makeArray(5)` and then `delete [] p`, compiles cleanly with `g++ -Wall -Wextra -std=c++11`. Changing it to `return a;` makes `g++` warn address of local variable 'a' returned [-Wreturn-local-addr], because automatic memory dies with its function.

The 9/22 lecture shows the same contrast:

- `int arr1[100];` has a fixed size and cannot grow.
- `int *arr2 = new int[100];` can later be replaced by a bigger `new int[200]`. This is the idea behind the self-sizing array (vector).

الشرح بالعربي خطوة بخطوة

عندنا ثلاث طرق لحجز الذاكرة، والفرق بينها يتضح بثلاثة أسئلة: أين تُحجز؟ متى تُنشأ؟ ومن يحررها؟

(١) الحجز التلقائي: يشمل المتغيرات المحلية داخل الدالة، ومعاملات الدالة، والمصفوفة المحلية مثل

```
int arr1[10];
```

يوضع في إطار الدالة على المكس. يُنشأ تلقائياً عند استدعاء الدالة ويختفي تلقائياً عند خروجها، فلا تحتاج أن تحرره بنفسك. لكن حجمه لازم يكون معروفاً وقت الترجمة، ومساحة المكس محدودة.

(٢) الحجز الديناميكي: تطلب الذاكرة أثناء التشغيل من الكومة باستخدام

```
new
```

وتستطيع أن تحدد الحجم وقت التشغيل، مثلاً رقم يُدخله المستخدم. تبقى الذاكرة موجودة حتى بعد انتهاء الدالة التي حجزتها، إلى أن تحررها أنت بنفسك:

```
delete [] p;
```

إذا نسيت التحرير يحدث تسريب الذاكرة. ولا تصل إلى هذه الذاكرة إلا عن طريق مؤشر، والمؤشر نفسه يكون غالباً متغيراً محلياً على المكس.

٣) الحجز الساكن (للمقارنة فقط): يشمل المتغيرات العامة والمتغيرات الساكنة داخل الدوال. تُحجز مرة واحدة عند تحميل البرنامج في منطقة المتغيرات العامة، وتبقى طول عمر البرنامج. انتبه: كلمة ساكن هنا لا تعني أن القيمة لا تتغير، بل تعني أن مكانها في الذاكرة ثابت طوال التشغيل.

فكرة للحفظ:

- التلقائي: المكس، ويتحرر وحده.
- الديناميكي: الكومة، وأنت المسؤول عن تحريره.
- الساكن: المنطقة العامة، ويعيش طول البرنامج.

Lecture link: 8/20-24 memory diagram: the STACK frame holds l1, c1 and the local array arr1 (automatic). The HEAP block is reached through 'ptr', which lives on the stack (dynamic). GLOBALS are the static area. hello2.cpp contrasts int arr1[10] (automatic) with aptr = new int[10]; ... delete [] aptr; (dynamic). Instructor's sort.cpp: loadData() does new int[n] and returns the pointer, with the comment 'allocates memory in heap, user must deallocate'. 9/22: fixed-size int arr1[100] vs int *arr2 = new int[100].

G3. What is a pointer in C++

Answer

A **pointer** is a variable whose value is a **memory address**. It holds the address of another variable or object, or `nullptr`, which means it points to nothing.

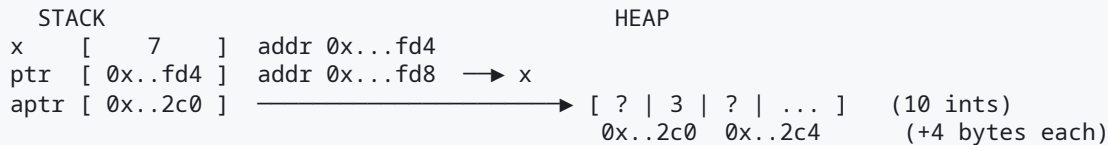
- **Declaration** uses `*`. In `int *ptr;`, `ptr` holds the address of an `int`.
- **Operators:**
 - `&x` (address-of) gives the address of `x`.
 - `*ptr` (dereference) reads or writes the value stored at that address.
 - `p->member` is the same as `(*p).member`, e.g. `s->print()`.
- **The type matters.** It tells the compiler what is stored at the address and how far pointer arithmetic moves. `aptr + 1` advances by `sizeof(int)` bytes (4 on our machines), and `aptr[i]` means `*(aptr + i)`.
- **A pointer is itself a variable**, with its own storage (8 bytes on a 64-bit system) and its own address, so `&aptr` is not the same as `aptr`.
- **Uses:**
 - It is the only way to reach **heap** memory from `new`.
 - A function can change the caller's data when it is passed an address.
 - Arrays: an array name acts as the address of element 0.
 - Objects: `Student *s = new Student;` and the `this` pointer.
 - Generic code with `void *`, which is just an address and must be cast before use.
- **Dangers:** dereferencing an uninitialized pointer, a `nullptr`, or a dangling pointer (after `delete`) is **undefined behavior**. It often crashes the program with a segmentation fault.

```

int main() {
    int x = 5;
    int *ptr = &x;           // ptr holds the address of x
    *ptr = 7;                // x is now 7
    int *aptr = new int[10]; // aptr (stack) -> 10 ints (heap)
    aptr[1] = 3;             // same as *(aptr + 1) = 3
    delete [] aptr;
    aptr = nullptr;
}

```

Worked steps / details



Tested run (g++ -Wall -Wextra -std=c++11; the addresses change every run):

- `&aptr[0] = 0x5631dff682b0` and `&aptr[1] = 0x5631dff682b4` are 4 bytes apart, which is one `int`.
- `ptr (0x7ffe81240a44)` and `&ptr (0x7ffe81240a48)` are different addresses, because the pointer is its own variable.

From `hello2.cpp`:

- `arr1` and `&arr1` print the same address, the start of the array. They have different types, though: `int*` after decay, and `int (*)[10]`.
- `aptr` is the heap address it stores. `&aptr` is where the pointer itself lives on the stack.

`void *` in the 9/10 generic sort: `int *a1 = (int*)a;`. A void pointer is just an address, and it cannot be dereferenced until it is cast to a real type.

الشرح بالعربي خطوة بخطوة

المؤشر متغير له اسم ومكان في الذاكرة مثل أي متغير، لكن القيمة المخزنة فيه ليست بيانات عادية مثل ٥ أو ٧، بل عنوان في الذاكرة يدلّك على مكان متغير آخر.

تخيل الذاكرة شارعاً فيه بيوت، ولكل بيت رقم وهو العنوان. المتغير العادي هو البيت وما بداخله، والمؤشر ورقة مكتوب عليها رقم بيت.

العمليات الأساسية:

- علامة العنوان تعطيك عنوان المتغير، ونخزنه في المؤشر:

```
ptr = &x;
```

- علامة النجمة قبل المؤشر معناها: اذهب إلى هذا العنوان، واقرأ القيمة هناك أو اكتبها. فهذا السطر يغير قيمة المتغير الأصلي نفسه:

`*ptr = 7;`

- نوع المؤشر مهم. المؤشر إلى عدد صحيح يعرف أن العنصر التالي يبعد بحجم عدد صحيح واحد (4 بايت على أجهزتنا)، لذلك هذا التعبير يشير إلى العنصر التالي وليس إلى البايت التالي:

`aptr + 1`

لماذا نحتاجه؟

- الذاكرة المحجوزة في الكومة ليس لها اسم، والطريقة الوحيدة للوصول إليها هي المؤشر الذي يرجعه

`new`

- نمرر العنوان إلى دالة حتى تستطيع تعديل بيانات من استدعاها.
- اسم المصفوفة يعمل كعنوان أول عنصر فيها.

انتبه:

- المؤشر نفسه له مكان في الذاكرة وعنوان خاص به. هذا هو الفرق في مثال الأستاذ بين قيمة المؤشر (عنوان الكتلة في الكومة) وعنوان المؤشر (مكانه في المكس).
- لا تستخدم مؤشراً بدون قيمة ابتدائية، أو مؤشراً فارغاً، أو مؤشراً إلى ذاكرة تم تحريرها. النتيجة سلوك غير محدد، وغالباً ينهار البرنامج.

Lecture link: 8/20-24 hello2.cpp prints `aptr` vs `&aptr`, `&aptr[0]` vs `&aptr[1]`, and `arr1` vs `&arr1`. The memory diagram shows 'ptr' on the stack pointing into the heap. 9/10 generic C sort: void pointers ('void ptr = just an address', cast with (int)a). 9/15-17: Student *s = new Student; s->print(); and the 'this' pointer (this == &s).

G4. How are pointers different from references in C++

Answer

A **pointer** is a separate variable that stores an address. A **reference** is an **alias**: another name for a variable that already exists.

	Pointer <code>int *p</code>	Reference <code>int &r = x</code>
Initialization	may be declared uninitialized or <code>nullptr</code>	must be initialized when declared
Null	can be <code>nullptr</code> (check before use)	there is no null reference; it is bound to an existing object when created (but it can still dangle if that object dies, e.g. a returned reference to a local)
Re-targeting	can point somewhere else later: <code>p = &y;</code>	bound for life; <code>r = y;</code> copies y's value into x
Syntax	needs <code>&x</code> to get the address and <code>*p</code> / <code>p-></code> to access	used exactly like the variable: <code>r</code> , <code>r.field</code>
Memory	a real object with its own storage and address (<code>&p</code> differs from <code>p</code>)	not a separate object you can reach: <code>&r == &x</code> (the compiler may still use a hidden pointer inside, e.g. for a reference parameter)
Arithmetic	<code>p++</code> , <code>p + i</code> , <code>p[i]</code> move through memory	no address arithmetic; <code>r++</code> just adds 1 to x
Other	pointer-to-pointer, arrays of pointers, holds <code>new</code> results	no reference-to-reference, no arrays of references

When to use which:

- Use **references** for function parameters. Nothing is copied, and the callee can change the caller's variable. Examples from the course:
 - The instructor's `sort.cpp` has `void swap(int &a, int &b)`, called as `swap(arr[j], arr[j+1]);`.
 - Use `const T&` for large read-only objects, as in `comp(const T&, const T&)` (`sortt.cpp`).
 - `std::ostream& print(std::ostream&)` in the Student class.
 - The copy constructor `Student(Student &)` (usually written `Student(const Student &)`).
- Use **pointers** when you need `nullptr`, re-targeting, arrays and pointer arithmetic, or dynamic memory (`new/delete`).

```
#include <iostream>
void incP(int *p) { (*p)++; }    // call: incP(&x);
void incR(int &r) { r++; }      // call: incR(x);

int main() {
    int x = 10, y = 20;
    int *p = &x; p = &y;        // pointer re-targeted to y
    int &r = x; r = y;          // x becomes 20; r still refers to x
    incP(&x);                   // x = 21
    incR(x);                   // x = 22
    std::cout << x << " " << *p << std::endl;    // prints 22 20
}
```

Worked steps / details

Trace (compiled with `g++ -Wall -Wextra -std=c++11` and run):

```
int main() {
    int x = 10, y = 20;
    int *p = nullptr;    // p points nowhere
```

```

p = &x;           // p -> x
*p = 11;          // x = 11
p = &y;           // p re-targeted -> y
int &r = x;        // r is another name for x
r = 12;           // x = 12
r = y;           // x = 20 (value copy; r still refers to x)
incP(&x);         // x = 21 (pass by pointer: caller passes the address)
incR(x);          // x = 22 (pass by reference: caller passes the variable)
}
// output: x=22 y=20 *p=20, and &r == &x (same address)

```

Memory picture:

```

x [22] <— r (one box with two names)
y [20] <— p [addr of y] (p is its own box, 8 bytes)

```

Tie-in to the 9/22 lecture:

- Student *s4 = s3; copies only the pointer, so both point to the same heap object.
- A copy constructor must take its parameter by reference, as in Student(Student &). If it took Student by value, creating that parameter would itself need a copy, which means another copy-constructor call, forever. g++ actually rejects it with invalid constructor; you probably meant 'Student (const Student&)'. .

الشرح بالعربي خطوة بخطوة

المرجع اسم ثانٍ لمتغير موجود أصلاً، مثل اللقب: شخص واحد باسمين. أما المؤشر فمتغير مستقل له مكانه الخاص ويحمل عنواناً.

الفروق المهمة، احفظها كنقاط:

(١) التهيئة: المرجع لازم يرتبط بمتغير لحظة تعريفه. المؤشر يمكن تعريفه بدون قيمة، أو بالقيمة الفارغة:

nullptr

(٢) القيمة الفارغة: لا يوجد مرجع فارغ. المؤشر قد يكون فارغاً، لذلك نفحصه قبل استخدامه. لكن انتبه: إذا انتهى عمر المتغير الأصلي يصبح المرجع معلقاً، لذلك لا ترجع مرجعاً لمتغير محلي.

(٣) تغيير الهدف: المؤشر نستطيع أن نجعله يشير إلى متغير آخر لاحقاً. المرجع يبقى مرتبطاً بنفس المتغير طول عمره، وإذا أسندت له قيمة فأنت تنسخ القيمة إلى المتغير الأصلي، ولا تغير الارتباط.

(٤) طريقة الاستخدام: مع المؤشر تحتاج علامة النجمة للوصول إلى القيمة، وعلامة العنوان لأخذ العنوان. أما المرجع فتستخدمه كأنه المتغير نفسه.

(٥) الذاكرة: المؤشر له عنوانه الخاص المختلف عن عنوان المتغير. المرجع ليس له عنوان خاص تستطيع أخذه، فعنوانه هو عنوان المتغير الأصلي نفسه. قد يستخدم المترجم مؤشراً مخفياً في الداخل، لكنك لا تستطيع الوصول إليه.

٦) المؤشر يسمح بحساب العناوين (الانتقال إلى العنصر التالي)، ويمكن عمل مؤشر إلى مؤشر ومصفوفة من المؤشرات. المرجع لا يسمح بذلك، وزيادة المرجع بواحد تزيد قيمة المتغير الأصلي فقط. متى أستخدم أيهما؟

- المرجع ممتاز لتمثيل المعاملات إلى الدوال بدون نسخ، مع إمكانية تعديل الأصل. مثال الأستاذ في برنامج الفرز دالة التبادل:

```
void swap(int &a, int &b)
```

- وكذلك دالة الطباعة في صف الطالب تستقبل مرجعاً لمجرى الإخراج، ومنشئ النسخ يجب أن يستقبل مرجعاً.
- المؤشر عندما نحتاج ذاكرة ديناميكية، أو قيمة فارغة، أو تغيير الهدف.

Lecture link: Instructor's sort.cpp: void swap(int &a, int &b), with the comment 'int& reference to first value', called from bubbleSort as swap(arr[j], arr[j+1]). 9/15 template function-pointer version int (*comp)(const T&, const T&). 9/17 Student example: std::ostream& print(std::ostream&) takes and returns a reference. 9/22: copy constructor Student(Student &) (deep copy), and Student *s4 = s3; copies only the pointer, so both refer to the same object. 8/20-24 hello2.cpp shows the pointer aptr with its own address &aptr.

G5. What is a memory leak, explain with code

Answer

A **memory leak** happens when memory allocated on the **heap** with `new` / `new[]` is never released with `delete` / `delete []`, and the program loses the last pointer to it. The pointer either goes out of scope or is overwritten.

- The block stays reserved while the program runs, and the OS reclaims it only when the process exits. Nothing can use it or free it in the meantime.
- In a loop or a long-running program the leaks add up. Memory use keeps growing, the program slows down, and `new` can eventually fail (`std::bad_alloc`).
- Only dynamic (heap) memory can leak. Stack variables are freed automatically.

Leak (resizing a self-sizing array, 9/22 lecture; the code is inside a function):

```
int *arr2 = new int[100];           // 100 ints on the heap
// ... array is full, grow it to 200
int *temp = arr2;                   // temp -> old block
arr2 = new int[200];                // arr2 -> new, bigger block
for (int i = 0; i < 100; i++)       // copy data from temp to arr2
    arr2[i] = temp[i];
// forgot: delete [] temp; -> old 100-int block is never de-allocated
// once temp goes out of scope its address is lost = MEMORY LEAK
```

Fixed:

```
int *temp = arr2;
arr2 = new int[200];
for (int i = 0; i < 100; i++)
    arr2[i] = temp[i];
delete [] temp;           // return the old block to the heap
temp = nullptr;          // no dangling pointer
// ... later, when finished with the array:
delete [] arr2;
arr2 = nullptr;
```

Rules to avoid leaks:

- Pair every `new` with one `delete`, and every `new[]` with one `delete []`.
- Free memory that a class owns in its **destructor**.
- Or let `std::vector`, `std::string` or `std::unique_ptr` manage the memory.

Worked steps / details

A second classic example is a leak inside a function:

```
void leaky(int n) {
    int *p = new int[n];    // block on the HEAP; p itself is on the STACK
    p[0] = 1;
}                           // frame popped -> p gone, block never deleted -> LEAK

void fixed(int n) {
    int *p = new int[n];
    p[0] = 1;
    delete [] p;           // freed before p disappears
    p = nullptr;
}
```

What happens in memory (resize example):

before the delete:		without delete [] temp, after temp is gone:	
STACK	HEAP	STACK	HEAP
arr2 [0]	→ [200 ints]	arr2 [0]	→ [200 ints]
temp [0]	→ [100 ints]		[100 ints] <- no pointer to it: LEAKED

Verified with `valgrind (g++ -Wall -Wextra -std=c++11, all compile cleanly):`

- Calling `leaky(1000)` 3 times: `valgrind --leak-check=full` reports definitely lost: 12,000 bytes in 3 blocks (3 x 1000 x 4 bytes).
- The resize code without `delete [] temp`: definitely lost: 400 bytes in 1 blocks, which is the old 100-int block.
- The fixed resize code: All heap blocks were freed -- no leaks are possible.
- `g++ -fsanitize=address` also reports 12000 byte(s) leaked in 3 allocation(s) when the program exits.

Ownership tie-in: the instructor's `loadData()` in `sort.cpp` says "allocates memory in heap, user must deallocate". Whoever receives the returned pointer owns the block and must eventually call `delete [] data;`.

Related bugs from lecture:

- **Use after deallocation:** reading `temp[0]` after `delete [] temp` is undefined behavior.
- **Double delete.**
- **Mismatched delete:** memory from `new[]` must be freed with `delete []`, not plain `delete`.

الشرح بالعربي خطوة بخطوة

تسريب الذاكرة يحدث عندما تحجز ذاكرة في الكومة، ثم تفقد عنوانها قبل أن تحررها.

لماذا هذا خطير؟

- الكومة لا تنظف نفسها بنفسها.
- المؤشر الذي يحمل العنوان يعيش عادة على المكدمس. عندما تنتهي الدالة يختفي المؤشر، لكن الكتلة في الكومة تبقى محجوزة ولا أحد يعرف مكانها، فلا يمكن استخدامها ولا تحريرها. نظام التشغيل يستعيدها فقط عند انتهاء البرنامج.
- إذا تكرر ذلك داخل حلقة، أو في برنامج يعمل مدة طويلة، يزيد استهلاك الذاكرة حتى يبطؤ البرنامج أو يفشل الحجز.

مثال الأستاذ من محاضرة المصفوفة ذاتية الحجم: عندنا مصفوفة من 100 عنصر ونريد تكبيرها إلى 200.

(١) نحفظ العنوان القديم في مؤشر مؤقت.

(٢) نحجز كتلة جديدة أكبر.

(٣) ننسخ البيانات من الكتلة القديمة إلى الجديدة.

(٤) نحرر الكتلة القديمة:

```
delete [] temp;
```

إذا نسيت الخطوة الأخيرة، تبقى الكتلة القديمة محجوزة إلى نهاية البرنامج، وهذا هو التسريب.

القواعد الذهبية:

- لكل حجز تحرير واحد مقابل له. حجز المصفوفة يقابله التحرير مع الأقواس المربعة، وحجز العنصر الواحد يقابله التحرير العادي.
- بعد التحرير اجعل المؤشر فارغاً، حتى لا تستخدم ذاكرة تم تحريرها.
- في الأصناف ضع التحرير داخل الهادم.
- والأسهل من كل ذلك أن تستخدم المتجه، لأنه يحرر ذاكرته بنفسه:

```
std::vector
```

Lecture link: 9/22 self-sizing array (vector): `int *arr2 = new int[100]; int *temp = arr2; arr2 = new int[200];` copy data from temp to arr2; delete [] temp; with the note 'if missing, the old space is not deallocated - memory leak', plus the bug 'use after deallocation'. 8/20-24 hello2.cpp ends with `delete [] aptr; aptr = nullptr;`. Instructor's sort.cpp loadData(): 'allocates memory in heap, user must deallocate'.

G6. What is the program stack and what is it's use

Answer

The **program stack** (call stack) is the R/W memory region at the **top** of the process's memory, as in the lecture diagram. It manages **function calls** and **grows down**, toward the heap.

- **Push:** each function call pushes a **stack frame** (activation record). The frame holds:
 - the function's **parameters**
 - the **return address**, where the caller continues
 - saved registers and the caller's frame pointer
 - the function's **automatic local variables, constants and local arrays**. For example, main's frame holds `l1`, `c1`, `arr1[10]` and the pointer `aptr`.
- **fp** (frame pointer) marks the top of the current frame. **sp** (stack pointer) marks the bottom, which is the current end of the stack.
- **Pop:** when the function returns, its frame is popped and sp moves back up. The locals are destroyed automatically, and execution resumes at the return address.
- The LIFO (last in, first out) order matches nested calls. Each recursive call gets its own frame and its own copy of the locals.

Uses:

- automatic allocation and deallocation of locals, which is very fast because it only moves sp
- passing parameters and return values
- remembering where to return
- supporting recursion

Limits:

- The stack has a limited, fixed size (8 MB by default on Linux; `ulimit -s` shows 8192 KB). Infinite or very deep recursion, or huge local arrays, cause a **stack overflow**.
- Writing past the end of a local array overwrites other frame data, such as the return address. This is **stack smashing**.
- Never return the address of a local variable, because it will dangle.

Worked steps / details

```
#include <iostream>
int fact(int n) {                // each call gets its OWN stack frame
    int local = n;
    if (n <= 1) return 1;
    return local * fact(n - 1);  // pushes a new frame BELOW this one
}
```

```

}
int main() { int x = 4; std::cout << fact(x); } // prints 24

```

The stack at its deepest point:

```

high addresses (Top)
+-----+ <- fp(main)
| main:  x = 4 |
+-----+
| fact(4): n=4, local=4, ret addr |
+-----+
| fact(3): n=3, local=3, ret addr |
+-----+
| fact(2): n=2, local=2, ret addr |
+-----+ <- fp (current frame)
| fact(1): n=1, local=1, ret addr |
+-----+ <- sp
      | stack grows DOWN
      v
      free space
      ^
      | heap grows UP
low addresses

```

The calls then unwind: fact(1) returns 1 and its frame is popped, then fact(2) returns 2, fact(3) returns 6, and fact(4) returns 24. sp moves back up each time.

Tested run (g++ -Wall -Wextra -std=c++11, printing &x and &local):

- main &x = 0x7ffc0c596814
- fact(4) &local = ...67f4
- fact(3) ...67c4
- fact(2) ...6794
- fact(1) ...6764

Each new frame is at a **lower** address (48 bytes lower here), which shows the stack grows down.

Stack smashing demo:

```

int main() {
    int arr1[10];
    for (int i = 0; i <= 20; i++) arr1[i] = i; // writes past the end of arr1
    return arr1[0];
}

```

With the default Ubuntu g++, this aborts at run time with ***** stack smashing detected ***: terminated**.

الشرح بالعربي خطوة بخطوة

المكدس منطقة في أعلى ذاكرة البرنامج، للقراءة والكتابة، ووظيفته إدارة استدعاءات الدوال.

كيف يعمل؟ فُكّر فيه كرصة صحن: آخر صحن تضعه هو أول صحن تأخذه.

- عندما تُستدعى دالة، يُضاف لها إطار جديد في المكس. في الإطار: معاملات الدالة، وعنوان الرجوع (أين يكمل البرنامج بعد انتهاء الدالة)، والمتغيرات والثوابت والمصفوفات المحلية.
 - المكس ينمو إلى الأسفل، باتجاه الكومة.
 - مؤشر الإطار يحدد أعلى إطار الدالة الحالية:
- fp
- ومؤشر المكس يحدد أسفل الإطار، أي النهاية الحالية للمكس:
- sp
- عندما تنتهي الدالة يُزال إطارها ويرجع مؤشر المكس إلى الأعلى، فتختفي متغيراتها المحلية تلقائياً. وهذا بالضبط هو الحجز التلقائي.

فوائده:

- حجز المتغيرات المحلية وتحريرها بسرعة وبشكل تلقائي، لأنه يكفي تحريك مؤشر المكس.
- تمرير المعاملات.
- حفظ عنوان الرجوع.
- دعم الاستدعاء الذاتي، لأن كل استدعاء له إطار خاص ونسخة خاصة من المتغيرات.

مشاكله:

- حجمه محدود. الاستدعاء الذاتي الذي لا ينتهي، أو المصفوفة المحلية الضخمة، يسببان فيضان المكس.
 - الكتابة بعد نهاية مصفوفة محلية تخرب بيانات الإطار وعنوان الرجوع. هذا ما كتبه الأستاذ بالأحمر: تحطيم المكس.
- وقد يوقف البرنامج نفسه عند التشغيل برسالة:

stack smashing detected

Lecture link: 8/20-24 memory diagram: STACK at the top (R/W), with an arrow showing it grows DOWN. main's 'stack frame' holds parameters, variables/constants (l1, c1) and a local array (arr1). fp = frame pointer at the top of the frame, sp = stack pointer at the bottom. 'Stack smashing' is marked in red where writes run past the local array. The heap below grows up.

G7. Give an example of allocating a variable in global memory space, stack memory space and heap memory space

Answer

Based on the instructor's hello2.cpp:

```

#include <iostream>
using namespace std;

int g1 = 23;           // GLOBAL space (globals, R/W): whole program lifetime
const int g2 = 23;     // global constant: read-only (g. const, R/O) area

void counter() {
    static int calls = 0; // static local: stored with the GLOBALS, not on the stack
    calls++;             // initialized once, keeps its value between calls
    cout << "calls = " << calls << endl;
}

int main() {
    int l1 = 12;         // STACK: automatic local in main's stack frame
    const int c1 = 14;   // STACK: local constant in main's frame
    int arr1[10];        // STACK: local array in main's frame
    arr1[0] = l1;

    int *aptr = nullptr; // STACK: the pointer variable itself
    aptr = new int[10];  // HEAP: block of 10 ints that aptr points to
    aptr[0] = c1;

    counter(); counter(); // prints calls = 1, then calls = 2

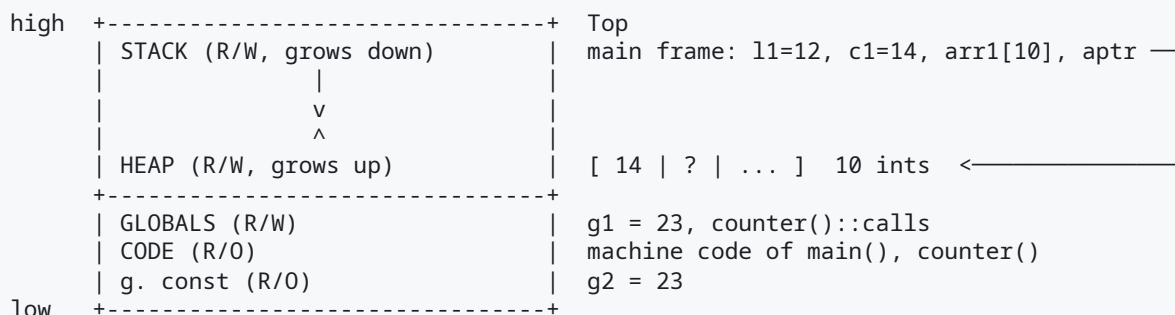
    cout << &g1 << " " << &g2 << " " << &l1 << " " << arr1 << " "
         << &aptr << " " << aptr << endl;

    delete [] aptr;      // free the heap block (else memory leak)
    aptr = nullptr;
    return 0;            // main's frame (l1, c1, arr1, aptr) popped automatically
}

```

- **Global:** `g1` is in the global area. `g2` is in the read-only constant area, and the static local `calls` is in the global/static area. They are created at program start and live until the program exits.
- **Stack:** `l1`, `c1`, `arr1` and the pointer `aptr` live in main's stack frame. They are freed automatically when main returns.
- **Heap:** the 10-int block from `new int[10]`. It lives until `delete [] aptr` and can only be reached through the pointer.

Worked steps / details



Tested run of an address-printing version (`g++ -Wall -Wextra -no-pie`, so the addresses are easy to read; they change between runs):

```

&g1      = 0x404048      (global)
&g2      = 0x402004      (global const)

```

```

&l1      = 0x7ffe485a3ee0 (stack)
arr1     = 0x7ffe485a3ef0 (stack)
&aptr    = 0x7ffe485a3ee8 (stack)
aptr     = 0x1358a2b0      (heap)
&aptr[1] = 0x1358a2b4      (heap, 4 bytes later)

```

Running `nm -C` on the executable of the program above shows:

- `D g1`: initialized R/W data
- `r g2`: read-only data
- `b counter()::calls`: zero-initialized static data (.bss)

This confirms that `g2` is read-only and that the static local lives with the globals, not on the stack.

Stack addresses (0x7ffe...) are very high, while the heap and the globals are much lower. This matches the lecture diagram: the stack is at the top, and the heap is above the globals. Writing `g2 = 5;` would be a compile error, because `g2` is `const`.

الشرح بالعربي خطوة بخطوة

المطلوب مثال واحد يوضح المناطق الثلاث، ونستخدم مثال الأستاذ.

(١) المنطقة العامة: أي متغير تعرّفه خارج كل الدوال يعيش طول عمر البرنامج في منطقة المتغيرات العامة (للقراءة والكتابة)،
مثل:

```
int g1 = 23;
```

- الثابت العام يوضع في منطقة للقراءة فقط.
- المتغير الساكن داخل دالة يعيش هنا أيضاً وليس في المكس، ولذلك يحتفظ بقيمته بين الاستدعاءات. في المثال يطبع واحداً ثم اثنين.

(٢) المكس: أي متغير محلي داخل الدالة الرئيسية، ومعه المصفوفة المحلية. يُحجز في إطار الدالة ويختفي عند خروجها،
مثل:

```
int l1 = 12;
```

(٣) الكومة: هذا السطر يحجز كتلة من عشرة أعداد صحيحة في الكومة:

```
aptr = new int[10];
```

انتبه لهذه النقطة الذكية: المؤشر نفسه متغير محلي، أي أنه على المكس، لكنه يشير إلى كتلة في الكومة. وفي النهاية نحرر الكتلة ونجعل المؤشر فارغاً حتى لا يحدث تسريب.

إذا طبعت العناوين ستلاحظ:

- عناوين المكس كبيرة جداً، لأنه في أعلى الذاكرة.

- عناوين الكومة والمنطقة العامة أصغر بكثير.
- كل عنصرين متتاليين في كتلة الكومة يفصل بينهما 4 بايت.

Lecture link: Directly based on the instructor's hello2.cpp (8/20-24): int g1=23; const int g2=23; and in main: int l1=12; const int c1=14; int arr1[10]; int *aptr=nullptr; aptr=new int[10]; ... delete [] aptr; aptr=nullptr;. The program prints &g1, &g2, &l1, &c1, arr1/&arr1, aptr/&aptr, &aptr[0] and &aptr[1]. Region names and permissions follow the 8/20-24 memory diagram: STACK R/W growing down, HEAP R/W growing up, globals R/W, code R/O, g. const R/O.

Sorting

G8. Give the running time of the following with a reasonable explanation of why: a. Selection sort b. Insertion sort c. Mergesort d. Quicksort

Answer

Summary (N = number of elements)

Algorithm	Best	Average	Worst	Extra memory
Selection	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion	$O(N)$ (sorted data)	$O(N^2)$	$O(N^2)$ (reverse data)	$O(1)$
Mergesort	$O(N \log_2 N)$	$O(N \log_2 N)$	$O(N \log_2 N)$	$O(N)$ work array
Quicksort	$O(N \log_2 N)$	$O(N \log_2 N)$	$O(N^2)$	$O(\log N)$ stack (average)

a. Selection sort: $O(N^2)$ in every case. Each pass scans the *whole* unsorted part to find the max, then does 1 swap to move it to the end. Pass 1 makes N-1 comparisons, pass 2 makes N-2, and so on, so the total is $(N-1)+(N-2)+\dots+1 = N(N-1)/2 = N^2/2 - N/2$ comparisons, which is $O(N^2)$. It never checks whether the data is already sorted, so best = worst. It makes at most N-1 swaps, which is $O(N)$.

b. Insertion sort: best $O(N)$, average and worst $O(N^2)$. Each new element is compared with the sorted part from right to left. Larger elements shift one place right, and the inner loop **stops at the first element $\leq$ key**.

- Sorted input: 1 comparison and 0 shifts per element, so N-1 comparisons in total, which is **$O(N)$** .
- Reverse input: element i must pass all i elements before it, so $1+2+\dots+(N-1) = N(N-1)/2$ comparisons and shifts, which is **$O(N^2)$** .
- Random input: it goes about halfway back on average, so about $N^2/4$, which is **$O(N^2)$** .

c. Mergesort: $O(N \log_2 N)$ in every case. The array is halved again and again, which gives **$\log_2 N$ levels**. At each level, the merges together copy all N elements once. That is N work per level, so $N \times \log_2 N$ in total. As a recurrence: $T(N) = 2T(N/2) + cN$. The split depends only on positions, not on the data, so best = average = worst. The cost is an $O(N)$ extra work array.

d. Quicksort: best and average $O(N \log_2 N)$, worst $O(N^2)$. Partitioning costs $O(N)$ per level.

- If the pivot is close to the median, each partition splits into halves. That gives $\log_2 N$ levels, so **$O(N \log_2 N)$** .
- If the pivot is always the min or max (for example pivot = A[end] or A[start] on already sorted or reverse-sorted data), each split is (N-1, 0). That gives N levels and $(N-1)+(N-2)+\dots+1 = N(N-1)/2$ work, so **$O(N^2)$** . The recursion depth is also N.

Checked with a C++ program (N = 1024, comparisons; the random counts are from one random order and vary slightly from run to run):

- Selection sort did 523,776 comparisons on sorted, random **and** reverse data (= $N(N-1)/2$).
- Insertion sort did 1,023 comparisons on sorted data and 523,776 on reverse data.
- Mergesort did between 5,120 (sorted or reverse) and about 8,945 (random).
- Quicksort with pivot = A[end] did about 11,000 on random data but 523,776 on sorted data (recursion depth 1,023).

Worked steps / details

Derivations

- Selection: $C(N) = (N-1) + (N-2) + \dots + 1 = N(N-1)/2$. For N = 5 this is $4+3+2+1 = 10$ comparisons. Swaps $\leq N-1$.
- Insertion, best case (sorted): 1 comparison per element, so N-1 in total. Worst case (reverse): the element at index i is compared and shifted i times, so $1+2+\dots+(N-1) = N(N-1)/2$ comparisons **and** $N(N-1)/2$ shifts. Average case: about $N^2/4$.
- Mergesort: level k has 2^k pieces of size $N/2^k$, and merging all of them costs about N. There are $\log_2 N$ levels (N, N/2, ..., 1). Total: about $N \times \log_2 N$. Recurrence: $T(N) = 2T(N/2) + cN$ with $T(1) = c$, which gives $T(N) = cN \log_2 N + cN$.
- Quicksort, best case: $T(N) = 2T(N/2) + cN$, which is $O(N \log_2 N)$. Worst case: $T(N) = T(N-1) + cN = c(N + (N-1) + \dots + 1)$, which is $O(N^2)$.

Measured comparison counts (C++ program, N = 1024; $N(N-1)/2 = 523,776$, $N \log_2 N = 10,240$; the random row is one random order; median-of-3 counts are partition comparisons only)

Input	Selection	Insertion	Mergesort	Quicksort (pivot = A[end])	Quicksort (median-of-3)
sorted	523,776	1,023	5,120	523,776 (depth 1,023)	8,204 (depth 10)
random (one run)	523,776	260,140	8,945	10,962 (depth 19)	9,321 (depth 15)
reverse	523,776	523,776	5,120	523,776 (depth 1,023)	14,741 (depth 21)

الشرح بالعربي خطوة بخطوة

الفكرة العامة: نقيس زمن التشغيل بعدد العمليات الأساسية (مقارنة، نقل، تبديل) كلما كبر عدد العناصر. نرمز لعدد العناصر بالحرف ن.

(١) الترتيب بالاختيار: في كل مرور نبحث عن أكبر عنصر في الجزء غير المرتب ونبدله مع آخر خانة فيه، وهذه هي طريقة الدكتور في المحاضرة. البحث يمرّ على الجزء غير المرتب كله في كل مرة، ولا يستطيع أبداً أن يكتشف أن البيانات مرتبة

أصلاً. لذلك عدد المقارنات دائماً: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ ، أي أن الزمن من رتبة n^2 في أفضل حالة وأسوأ حالة والمتوسط. ميزته أنه يبذل مرة واحدة على الأكثر في كل مرور، أي $n-1$ تبديل فقط.

(٢) الترتيب بالإدراج: نأخذ العنصر التالي، ونزيع كل عنصر أكبر منه في الجزء المرتب خانة إلى اليمين، ثم نضعه في الفراغ. الحلقة الداخلية تتوقف فوراً عندما تجد عنصراً أصغر منه أو يساويه.

- بيانات مرتبة: مقارنة واحدة لكل عنصر وبدون إزاحة، فالمجموع $n-1$ ، أي رتبة n (أفضل حالة).
- بيانات معكوسة: كل عنصر يمرّ على كل العناصر التي قبله، فالمجموع $n(n-1)/2$ ، أي رتبة n^2 (أسوأ حالة).
- بيانات عشوائية: في المتوسط يمرّ على نصف ما قبله، أي تقريباً $n/4$ ، وهذه أيضاً رتبة n^2 .

(٣) الترتيب بالدمج: نقسم المصفوفة نصفين، ثم نقسم كل نصف نصفين، وهكذا حتى يبقى في كل جزء عنصر واحد. عدد مستويات التقسيم يساوي لوغاريتم n للأساس 2. في كل مستوى يمرّ الدمج على كل العناصر مرة واحدة، أي أن عمل المستوى الواحد مقداره n . إذن الزمن $= n \times \log_2 n$ في كل الحالات، لأن التقسيم يعتمد على المواقع وليس على القيم. عيبه أنه يحتاج مصفوفة عمل إضافية بحجم n .

(٤) الترتيب السريع: التقسيم حول المحور يكلف n في كل مستوى.

- إذا كان المحور قريباً من الوسيط، ينقسم الجزء إلى نصفين، فيكون عدد المستويات $\log_2 n$ والزمن $n \times \log_2 n$ (أفضل حالة والمتوسط).

- إذا كان المحور دائماً أصغر عنصر أو أكبر عنصر (مثلاً نختار آخر عنصر والبيانات مرتبة أو معكوسة)، يصبح أحد الجزأين فارغاً والآخر فيه $n-1$ عنصراً، فيكون عدد المستويات n والزمن $n(n-1)/2$ ، أي رتبة n^2 (أسوأ حالة).

في ورقة الامتحان تُكتب الرتبة n^2 هكذا

$$O(N^2)$$

والرتبة $n \times \log_2 n$ هكذا

$$O(N \log_2 N)$$

تأكدنا بالبرنامج: مع 1024 عنصراً عمل الاختيار 523776 مقارنة، سواء كانت البيانات مرتبة أو عشوائية أو معكوسة. أما الإدراج فعمل 1023 مقارنة فقط مع البيانات المرتبة.

Lecture link: 8/27 bubble sort page: worst case $(N-1)+(N-2)+\dots+0 = N(N-1)/2$, so $O(N^2)$; best case $O(N)$ with early exit. 8/27 selection sort page: the instructor's find-max version, comparisons $\sim N^2$ so $O(N^2)$, swaps $\sim N$. 8/27 insertion sort page: shift larger elements right and drop the element into the hole. 9/1 mergesort page: 'Array of size $N \Rightarrow \log_2 N$ # of divisions', N work per level, so $O(N \log_2 N)$; 'A, Work' is the extra array. 9/3 quicksort page: best $O(N \log_2 N)$, worst $O(N^2)$, pivot = $A[\text{end}]$ (Lomuto) or $A[\text{start}]$ (Hoare).

G9. What is the difference between top-down and bottom-up Mergesort

Answer

Both use the **same merge step** (two sorted runs become one sorted run, using indices i, j, k). Both take **$O(N \log_2 N)$** time and need an **$O(N)$ work array**. They differ in **how the runs to be merged are produced**.

Top-down (recursive). This is the lecture's `merge_sort(A, start, end)`:

- Start from the whole array, compute `mid`, recursively sort `A[start..mid]` and `A[mid+1..end]`, then merge the two halves.
- Keep splitting in halves until each piece has 1 element (already sorted). The merging happens as the recursion returns.
- Because it is recursive, it puts function calls on the program stack, about $\log_2 N$ deep.

Bottom-up (iterative):

- No recursion and no split phase. Treat the array as N sorted runs of size 1.
- Pass 1 merges neighbouring runs of size 1 into runs of size 2. Pass 2 merges those into runs of size 4, then 8, 16, and so on, until there is one run of size N . That is $\text{ceil}(\log_2 N)$ passes, each $O(N)$.

```
void mergeSortBU(int A[], int N) {
    int *work = new int[N];
    for (int width = 1; width < N; width *= 2)           // run size 1,2,4,8,...
        for (int s = 0; s < N - width; s += 2 * width) {
            int mid = s + width - 1;
            int e = std::min(s + 2 * width - 1, N - 1); // last run may be shorter
            merge(A, work, s, mid, e);                  // merge A[s..mid] + A[mid+1..e]
        }
    delete [] work;
}
```

	Top-down	Bottom-up
Control	recursion	two nested loops (iterative)
Order of work	split down to size 1 first, then merge back up (depth-first)	merge level by level: 1 to 2 to 4 to 8 ...
Stack use	$O(\log N)$ call frames	no recursion
N not a power of 2	halves differ by at most 1	the last run may be shorter (it is merged with a full neighbour); a run with no partner in a pass is carried unchanged to the next pass
Time / memory	$O(N \log_2 N) / O(N)$	$O(N \log_2 N) / O(N)$

Both are stable (when the merge uses $\leq$) and give the same sorted result.

Worked steps / details

Example: 8 7 6 5 4 3 2 1 (checked with a program)

Bottom-up:

Pass	Run size	Array after the pass
1	1 to 2	[7 8] [5 6] [3 4] [1 2]
2	2 to 4	[5 6 7 8] [1 2 3 4]
3	4 to 8	[1 2 3 4 5 6 7 8]

Top-down on the same array: it splits into [8 7 6 5] [4 3 2 1], then [8 7] [6 5] [4 3] [2 1], then single elements. It then merges depth-first: [7 8], [5 6], then [5 6 7 8], then [3 4], [1 2], then [1 2 3 4], then the final merge. Both versions made 12 comparisons.

When N is not a power of 2, the groupings are different. Lecture array 5 4 8 1 3 6:

- Top-down: [5 4 8] [1 3 6], then [5 4] [8] and [1 3] [6], then [4 5 8] [1 3 6], then 1 3 4 5 6 8
- Bottom-up: [4 5] [1 8] [3 6], then [1 4 5 8] [3 6] ([3 6] has no partner at run size 2 and waits), then 1 3 4 5 6 8

الشرح بالعربي خطوة بخطوة

الطريقتان تستخدمان خطوة الدمج نفسها، أي دمج جزأين مرتبين في جزء واحد مرتب. لهما نفس الزمن $n \times \log_2 n$ ، وكلتاها تحتاج مصفوفة عمل بحجم n . الفرق في طريقة الوصول إلى الأجزاء التي ندمجها.

(١) الطريقة من الأعلى إلى الأسفل

top-down

هي الطريقة التي شرحها الدكتور، وفيها دالة تستدعي نفسها (استدعاء ذاتي). نبدأ بالمصفوفة كاملة ونحسب المنتصف، ثم نرتب النصف الأيسر، ثم النصف الأيمن، ثم ندمجها. يستمر التقسيم حتى يصبح كل جزء عنصراً واحداً، والعنصر الواحد مرتب تلقائياً. بعد ذلك يحدث الدمج ونحن راجعون من الاستدعاءات. هذه الاستدعاءات تُحفظ في المكس بعق يقارب $\log_2 n$.

(٢) الطريقة من الأسفل إلى الأعلى

bottom-up

لا يوجد فيها استدعاء ذاتي، فقط حلقات تكرار، ولا توجد مرحلة تقسيم. نعتبر كل عنصر مجموعة مرتبة طولها 1. في المرور الأول ندمج كل مجموعتين متجاورتين فتصبح المجموعات بطول 2، ثم بطول 4، ثم 8، حتى تصبح مجموعة واحدة بطول n .

مثال على المصفوفة:

8 7 6 5 4 3 2 1

بعد المرور الأول (مجموعات طولها 2):

7 8 | 5 6 | 3 4 | 1 2

بعد المرور الثاني (مجموعات طولها 4):

5 6 7 8 | 1 2 3 4

بعد المرور الثالث:

1 2 3 4 5 6 7 8

الخلاصة: الطريقة الأولى تعاودية، تقسم أولاً ثم تدمج وهي راجعة. الطريقة الثانية تكرارية، تدمج مستوى بعد مستوى من الأسفل. الثانية توفر كلفة استدعاء الدوال واستخدام المكس. وإذا لم يكن عدد العناصر من قوى العدد 2، فقد تكون المجموعة الأخيرة أقصر من غيرها فتدمج مع جارتها وهي أقصر، وقد لا تجد جارة في مرور ما فتنتقل كما هي إلى المرور التالي. في النهاية النتيجة واحدة.

Lecture link: 9/1 mergesort page: the recursive split diagram of 5 4 8 1 3 6 and the merge_sort(A, start, end) pseudocode (C = merge_sort(left), D = merge_sort(right), E = merge(C, D)) are the top-down version. The 9/1 merge algorithm (indices i, j, k, then copy the rest of A and the rest of B) is the same merge both versions use. Bottom-up does the merge levels of that diagram (the 'M' steps) with loops, from the bottom up.

G10. Explain the median-of-3 partitioning for Quicksort

Answer

Quicksort's partition should split the data around the **median** ('find median, partition around median'). Finding the true median is expensive, so we **guess** it and call the guess the pivot.

The simple guesses are `pivot = A[end]` (Lomuto) or `pivot = A[start]` (Hoare). On sorted or reverse-sorted data these are bad guesses: the pivot is the max or min, every split is (N-1, 0), and quicksort drops to $O(N^2)$.

Median-of-3: look at three elements: the **first** `A[start]`, the **middle** `A[(start+end)/2]` and the **last** `A[end]`. Use the **median of these three** as the pivot.

Lecture example: 8 1 12 3 6 9. First = 8, middle = 12, last = 9. In order they are 8, 9, 12, so the median is 9 and the pivot is 9. Partitioning around 9 gives 8 1 3 6 | 9 | 12.

Why it helps

- The pivot is never the smallest or the largest of the three, so at least one element ends up on each side of it. The extreme (N-1, 0) split can no longer happen.
- On **sorted** data the middle element is the true median of every subarray, so every split is a perfect half (recursion depth 10 for N = 1024). On **reverse-sorted** data the first split is a perfect half and the later splits stay well balanced (depth 21). Either way the time is $O(N \log_2 N)$ instead of $O(N^2)$.
- It is cheap: only 2 or 3 extra comparisons per partition.
- The worst case is still $O(N^2)$ in theory, with specially constructed inputs, but it is very unlikely.

Typical implementation: sort the three positions in place so that $A[start] \leq A[mid] \leq A[end]$, then move the median to the pivot position and partition as usual. For Lomuto, swap it with $A[end]$. The textbook version uses $A[end-1]$ and lets $A[start]$ and $A[end]$ act as sentinels.

```
int medianOf3(int A[], int start, int end) {
    int mid = (start + end) / 2;
    if (A[mid] < A[start]) swap(A[mid], A[start]);
    if (A[end] < A[start]) swap(A[end], A[start]);
    if (A[end] < A[mid]) swap(A[end], A[mid]); // A[start] <= A[mid] <= A[end]
    swap(A[mid], A[end]); // median -> A[end] (Lomuto pivot)
    return A[end];
}
// in quicksort: if (end > start) { medianOf3(A, start, end);
// p = partition(A, start, end); quicksort(A, start, p-1); quicksort(A, p+1, end); }
```

Worked steps / details

Trace on 8 1 12 3 6 9 (start = 0, end = 5, mid = 2), checked with a program:

1. The candidates are $A[0] = 8$, $A[2] = 12$ and $A[5] = 9$. The median is 9.
2. After medianOf3 the array is again 8 1 12 3 6 9, with pivot 9 at $A[end]$. (The third test swaps 12 and 9, then the last line swaps them back.)
3. Lomuto partition with pivot 9: the elements ≤ 9 (8, 1, 3, 6) move to the left, giving 8 1 3 6 9 12. The pivot index is $p = 4$. This matches the result on the lecture page.
4. Recurse on 8 1 3 6 and on 12.

Measured effect (N = 1024, Lomuto partition; partition comparisons only, not counting the 3 comparisons inside medianOf3; random = one random order)

Input	pivot = $A[end]$	median-of-3
sorted	523,776 comparisons, recursion depth 1,023	8,204 comparisons, depth 10
reverse	523,776, depth 1,023	14,741, depth 21
random (one run)	10,962, depth 19	9,321, depth 15

الشرح بالعربي خطوة بخطوة

الترتيب السريع يحتاج محوراً قريباً من الوسيط، أي القيمة التي تقع في المنتصف لو رتبنا البيانات، حتى ينقسم الجزء إلى نصفين متقاربين. لكن حساب الوسيط الحقيقي مكلف، لذلك نخمّنه.

التخمين البسيط هو أن نختار أول عنصر أو آخر عنصر محوراً. المشكلة أنه إذا كانت البيانات مرتبة أو معكوسة يكون المحور دائماً أصغر عنصر أو أكبر عنصر. عندها يصبح أحد الجزأين فارغاً، والجزء الآخر فيه كل العناصر ناقص واحد، فينهار الأداء إلى رتبة n^2 .

فكرة الوسيط من ثلاثة: ننظر إلى ثلاثة عناصر فقط هي الأول والأوسط والأخير، ونختار القيمة الوسطى بينها لتكون المحور.

مثال المحاضرة:

8 1 12 3 6 9

الأول 8، والأوسط 12، والأخير 9. إذا رتبنا الثلاثة تصبح 8 ثم 9 ثم 12، فالقيمة الوسطى هي 9، إذن المحور = 9. بعد التقسيم تكون العناصر الأصغر من 9 على يساره، والعنصر 12 على يمينه:

8 1 3 6 | 9 | 12

لماذا تنفع هذه الفكرة؟

- المحور لا يمكن أن يكون أصغر الثلاثة أو أكبرها، إذن يبقى عنصر واحد على الأقل في كل جهة منه، فلا يحدث التقسيم الأسوأ الذي يكون فيه أحد الجزأين فارغاً.
 - مع البيانات المرتبة يكون العنصر الأوسط هو الوسيط الحقيقي في كل جزء، فينقسم كل جزء نصفين بالضبط. ومع البيانات المعكوسة يكون التقسيم الأول نصفين بالضبط، ثم تبقى التقسيمات التالية متوازنة تقريباً. في الحالتين يصبح الزمن $\times$ لو₂ بدلاً من n^2 .
 - الكلفة الإضافية صغيرة: مقارنتان أو ثلاث في كل تقسيم.
 - أسوأ حالة ما زالت ممكنة نظرياً ببيانات مصممة خصيصاً، لكنها نادرة جداً.
- تحققنا بالبرنامج: مع 1024 عنصراً مرتباً، اختيار آخر عنصر محوراً أعطى 523776 مقارنة وعمق استدعاء 1023. أما الوسيط من ثلاثة فأعطى 8204 مقارنات وعمق 10 فقط. ومع 1024 عنصراً معكوساً أعطى الوسيط من ثلاثة 14741 مقارنة وعمق 21.
- طريقة التنفيذ المعتادة: نرتب الخانات الثلاث فيما بينها، ثم ننقل الوسيط إلى آخر الجزء (أو إلى الخانة التي قبل الأخيرة)، ثم نكمل التقسيم كالمعتاد.

Lecture link: 9/3 quicksort page: 'partition: find median, partition around median', then 'guess median (pivot)', with pivot = A[end] (Lomuto) or pivot = A[start] (Hoare). The page draws 'median-of-3' on the example 8 1 12 3 6 9, marks 8, 12 and 9, and picks 9. The page's Lomuto result is 8 1 3 6 9 12, which matches the trace here.

G11. What happens when the data is almost sorted for: a. Selection sort b. Insertion sort c. Mergesort d. Quicksort

Answer

a. Selection sort: no benefit, still $O(N^2)$. Every pass still scans the whole unsorted part to find the max, so it still makes $N(N-1)/2$ comparisons. Only the number of real swaps drops, because the max is often already at the end, so no swap is needed. The lecture counts this case as 0 swaps (P4). The runtime is about the same as for random data.

b. Insertion sort: its best case, about $O(N)$. Each new element is usually already $\geq$ the last element of the sorted part, or only a few places out. The inner loop stops after about 1 comparison and very few shifts are needed. The runtime is $O(N + \text{number of inversions})$, which is about $O(N)$. This is where insertion sort shines.

c. Mergesort: essentially unchanged, $O(N \log^2 N)$. It always splits in halves and copies every element at each of the $\log^2 N$ levels, whatever the order. It makes slightly fewer comparisons, because one half runs out early, but the data movement is the same. Optimization: skip the merge when $A[mid] \leq A[mid+1]$; then sorted data takes $O(N)$. A natural mergesort, which merges runs that already exist, gets the same benefit.

d. Quicksort: close to its worst case when the pivot is the first or last element. With $\text{pivot} = A[\text{end}]$ (or $A[\text{start}]$), the pivot is almost always the max (or min) of the subarray. The partitions are about $(N-1, 0)$, which gives about N levels of recursion. That means **$O(N^2)$** time and $O(N)$ recursion depth, with a risk of **stack overflow**. The fix is a **median-of-3** or **random pivot**, which brings it back to $O(N \log^2 N)$.

Summary: on almost-sorted data insertion sort is the best, selection sort and mergesort are not affected, and quicksort with a simple pivot is the worst.

Checked with a program ($N = 1024$, a sorted array with about 1% of neighbouring pairs swapped; the exact values vary slightly with which pairs are swapped):

- Selection: 523,776 comparisons.
- Insertion: 1,033 comparisons (versus about 260,000 on random data).
- Mergesort: 5,125 comparisons (versus about 8,945 on random data), with 10,240 moves in both cases.
- Quicksort with $\text{pivot} = A[\text{end}]$: 518,939 comparisons and depth 1,013. With median-of-3: 8,208 comparisons and depth 10.

Worked steps / details

Small example: 1 2 4 3 5 6 (almost sorted, $N = 6$, checked with a program)

- Insertion sort: inserting 2 and 4 takes 1 comparison each and no shift. Inserting 3 compares with 4 (shift), then with 2 (stop): 2 comparisons, 1 shift. Inserting 5 and 6 takes 1 comparison each. **Total: 6 comparisons, 1 shift**, which is about N .
- Selection sort on the same data still makes $5+4+3+2+1 = 15$ **comparisons** (1 real swap, in pass 3, when 4 moves to index 3).
- Quicksort with $\text{pivot} = A[\text{end}]$: pivot 6 is the max, so the split is $(5, 0)$. Pivot 5 is again the max, $(4, 0)$. Pivot 3 gives $(2, 1)$, and pivot 2 gives $(1, 0)$. That is **13 comparisons** in 4 levels, almost as bad as the 15 comparisons and 5 levels of fully sorted data.

الشرح بالعربي خطوة بخطوة

البيانات شبه المرتبة تعني أن معظم العناصر في مكانها الصحيح أو قريبة جداً منه.

أ) الترتيب بالاختيار: لا يستفيد شيئاً. في كل مرور ما زال يبحث عن الأكبر في الجزء غير المرتب كله، فيبقى عدد المقارنات $(N-1)/2$ كما هو. الذي يقل فقط هو عدد التبادلات، لأن الأكبر يكون غالباً في مكانه. الزمن يبقى رتبة N^2 .

ب) الترتيب بالإدراج: هذه أفضل حالة له. كل عنصر جديد يكون غالباً أكبر من آخر عنصر في الجزء المرتب، فتتوقف الحلقة الداخلية بعد مقارنة واحدة، والعناصر القليلة التي ليست في مكانها تتحرك خطوات قليلة. الزمن قريب من رتبة N . مثال

صغير:

احتاج الإدراج هنا 6 مقارنات وإزاحة واحدة، بينما احتاج الاختيار 15 مقارنة.

(ج) الترتيب بالدمج: تقريباً لا يتغير شيء. هو يقسم دائماً إلى نصفين وينسخ كل العناصر في كل مستوى، فيبقى الزمن $\times$ 2. يوجد تحسين بسيط: إذا كان آخر عنصر في النصف الأيسر أصغر من أول عنصر في النصف الأيمن أو يساويه، فلا حاجة للدمج أصلاً.

(د) الترتيب السريع: إذا كان المحور أول عنصر أو آخر عنصر، فهذه تقريباً أسوأ حالة له. يكون المحور غالباً أكبر عنصر أو أصغر عنصر في الجزء، فيأتي التقسيم غير متوازن: جزء فيه كل شيء تقريباً وجزء فارغ. يصبح عدد المستويات قريباً من n ، فالزمن رتبة n^2 ، والاستدعاءات عميقة جداً وقد يمتلئ المكسد. الحل هو الوسيط من ثلاثة أو اختيار محور عشوائي.

تحققنا بالبرنامج مع 1024 عنصراً شبه مرتب:

• الإدراج: 1033 مقارنة فقط.

• الاختيار: 523776 مقارنة.

• الدمج: 5125 مقارنة.

• السريع بمحور آخر عنصر: 518939 مقارنة وعمق 1013.

• السريع بالوسيط من ثلاثة: 8208 مقارنات وعمق 10.

الخلاصة: مع البيانات شبه المرتبة يكون الإدراج هو الأفضل، والترتيب السريع بمحور بسيط هو الأسوأ.

Lecture link: 8/27 selection sort page: comparisons are always $\sim N^2$, but the swap count can be 0 when the max is already in place (P4: '1 comp, 0 swap'). 8/27 insertion sort page: insert the next element into the sorted set by shifting, so few shifts when the data is almost sorted. 9/1 mergesort page: $\log_2 N$ levels $\times N$ work, whatever the data. 9/3 quicksort page: worst case $O(N^2)$ with pivot = $A[\text{end}]$ or $A[\text{start}]$, fixed by median-of-3.

G12. What is the main advantage/disadvantage of: a. Quicksort b. Mergesort c. Selection sort d. Insertion sort

Answer

Algorithm	Main advantage	Main disadvantage
a. Quicksort	Fastest in practice: average $O(N \log^2 N)$ with small constant factors. Sorts in place (no work array, only an $O(\log N)$ stack). Good cache locality.	Worst case $O(N^2)$ with a bad pivot, e.g. sorted data with a first or last pivot. Not stable . Recursive, so the recursion can get deep.
b. Mergesort	Guaranteed $O(N \log^2 N)$ in every case, so it is predictable. Stable . Good for linked lists and for sorting huge files (external sorting). Easy to parallelize.	Needs $O(N)$ extra memory for the work array and copies data back and forth. Gets no speed-up on already-sorted data (not adaptive).
c. Selection sort	Very simple. Fewest swaps (writes): at most $N-1$, so it is good when moving an element is expensive (large records, flash memory). In place, $O(1)$ memory.	Always $O(N^2)$: $N(N-1)/2$ comparisons even when the data is already sorted. Not stable.
d. Insertion sort	Simple, in place and stable. Adaptive: $O(N)$ on sorted or nearly-sorted data . Very fast for small N , which is why library sorts such as <code>std::sort</code> (introsort) and Timsort use it for small subarrays. Can sort data as it arrives (online).	$O(N^2)$ in the average and worst case. Reverse-sorted data needs $N(N-1)/2$ shifts, which is a lot of writes.

(Stable = elements with equal keys keep their original relative order.)

Worked steps / details

Supporting measurements (g++ -O2; times depend on the machine, the operation counts are exact)

- $N = 10,000$ ints in reverse order: selection made 49,995,000 comparisons and 5,000 swaps. Insertion made 49,995,000 comparisons and 49,995,000 shifts.
- $N = 4,000$ records of 512 bytes each (expensive to move):
 - reverse data: selection about 26 ms, insertion about 160 to 220 ms
 - random data: selection about 26 ms, insertion about 80 to 90 ms
 - sorted data: selection about 26 ms, insertion about 1 ms So selection's few writes win when moves are costly, and insertion wins on sorted or nearly-sorted data.
- $N = 1024$ sorted: quicksort with pivot = $A[\text{end}]$ made 523,776 comparisons ($O(N^2)$); mergesort made 5,120.

الشرح بالعربي خطوة بخطوة

أ) الترتيب السريع

الميزة: هو الأسرع عملياً في المتوسط ($n \times \log_2 n$ ثوابت صغيرة)، ويرتب داخل المصفوفة نفسها بدون مصفوفة إضافية. العيب: أسوأ حالة له رتبة n^2 إذا كان المحور سيئاً، مثل بيانات مرتبة مع اختيار أول عنصر أو آخر عنصر محوراً. وهو غير مستقر، والاستدعاء الذاتي فيه قد يصبح عميقاً.

ب) الترتيب بالدمج

الميزة: زمنه مضمون $n \times \log_2 n$ في كل الحالات، وهو مستقر، ومناسب للقوائم المترابطة والبيانات الضخمة المخزنة في

ملفات.

العيب: يحتاج ذاكرة إضافية بحجم n (مصفوفة العمل) وينسخ البيانات كثيراً، ولا يستفيد إذا كانت البيانات مرتبة.

ج) الترتيب بالاختيار

الميزة: بسيط جداً، وعدد التبادلات فيه أقل ما يمكن ($n-1$ على الأكثر)، فهو مناسب عندما يكون نقل العنصر مكلفاً مثل السجلات الكبيرة.

العيب: يعمل دائماً $n(n-1)/2$ مقارنة حتى لو كانت البيانات مرتبة، وهو غير مستقر.

د) الترتيب بالإدراج

الميزة: بسيط، ومستقر، ويعمل داخل المصفوفة نفسها. وهو سريع جداً مع البيانات شبه المرتبة (رتبة n) ومع المصفوفات الصغيرة، لذلك تستخدمه دوال المكتبات لترتيب الأجزاء الصغيرة.

العيب: رتبته n^2 في المتوسط وفي أسوأ حالة، ويحتاج إزاحات كثيرة جداً مع البيانات المعكوسة.

معنى «مستقر»: يحافظ على الترتيب الأصلي بين العناصر المتساوية، واسمه بالإنجليزية:

`stable`

ومعنى «داخل المصفوفة»: يرتب بدون ذاكرة إضافية كبيرة، واسمه بالإنجليزية:

`in place`

Lecture link: Summarizes the sorting pages: 8/27 (selection, insertion), 9/1 (mergesort and its extra 'A, Work' array) and 9/3 (quicksort best and worst case, pivot choice, median-of-3).

Objects, classes and vectors

G13. What are the main differences between a class and a struct in C++

Answer

Short answer: as types, `struct` and `class` in C++ are the *same* language feature. Both can have data members, member functions (methods), constructors/destructors, operator overloading, inheritance and virtual functions. There are only **two** differences, and both are about **defaults**:

	<code>struct</code>	<code>class</code>
Default access of members	public	private
Default inheritance access (decided by the keyword of the <i>derived</i> type)	public (<code>struct D : B</code> means <code>: public B</code>)	private (<code>class D : B</code> means <code>: private B</code>)

Difference in how they are used (convention, as in lecture):

- `struct` = "**primitive object**": a simple group of public data (a record), with little or no behavior.

- `class` = "**full object**": attributes + methods, information hiding (private / protected / public), constructor/destructor, inheritance, polymorphism.

```

struct Name {           // members public by default
    string first, last;
};

class Person {
    string ssn;           // private by default
public:
    string first, last;
    Person(string f, string l, string s) : ssn(s), first(f), last(l) {}
    void print() const { cout << first << " " << last << endl; }
};

int main() {
    Name n;
    n.first = "John";      // OK - public member
    Person p("John", "Doe", "123-45-6789");
    p.print();             // OK - public method
    // p.ssn = "000";      // compile ERROR - ssn is private
    return 0;
}

```

(Unlike C, a C++ struct can have methods, and you do not need to write `struct` again when you declare a variable: `Name n;`.)

Side note: only the keyword `class` (not `struct`) can be used in a template header such as `template <class T>`. That is a rule about the keyword, not a difference between the two kinds of type.

Worked steps / details

A struct can do everything a class can (constructor, methods):

```

struct Point {
    double x, y;
    Point(double a = 0, double b = 0) : x(a), y(b) {} // constructor
    double len2() const { return x*x + y*y; }        // method
};

```

Default inheritance access:

```

struct SD : Point {}; // = struct SD : public Point -> sd.x usable from main
class CD : Point {};  // = class CD : private Point -> cd.x NOT usable from main

```

(g++ for `cd.x = 1;` in main: error: 'double Point::x' is inaccessible within this context.)

Good practice: always write `public:` / `private:` and `: public Base` explicitly, so the defaults never matter.

الشرح بالعربي خطوة بخطوة

لنبدأ من الصفر: في لغة

C++

توجد كلمتان لصنع نوع جديد من الكائنات: البنية

struct

والصنف

class

.

الخطوة الأولى: افهم أن الاثنين قادران على الأشياء نفسها: متغيرات أعضاء، ودوال أعضاء (طرق)، ومُنشئ ومُدَمِّر ووراثه، ودوال افتراضية. إذن الفرق ليس في القدرة.

الخطوة الثانية: الفرق الحقيقي في اللغة هو «الإعداد الافتراضي للرؤية» فقط:

- في البنية: أي عضو لم تكتب له رؤية يكون عامًا تلقائيًا.
- في الصنف: أي عضو لم تكتب له رؤية يكون خاصًا تلقائيًا.

الخطوة الثالثة: نفس القاعدة في الوراثة، والمهم هنا كلمة النوع الابن: إذا كان الابن بنية فالوراثة عامة تلقائيًا، وإذا كان الابن صنفًا فالوراثة خاصة تلقائيًا.

الخطوة الرابعة: الفرق في طريقة الاستخدام (العُرف): الأستاذ سَمَّى البنية «كائنًا بدائيًا» يجمع بيانات مكشوفة، مثل بنية الاسم التي فيها الاسم الأول والأخير، وسَمَّى الصنف «كائنًا كاملاً» فيه بيانات مخفية وطرق ومُنشئ ومُدَمِّر ووراثه وتعدد أشكال.

مثال للفهم: مع بنية الاسم تستطيع أن تكتب في الدالة الرئيسية مباشرة

```
n.first = "John";
```

لأن العضو عام. أما في صنف الشخص فمحاولة الوصول إلى رقم الضمان الاجتماعي من الخارج تعطي خطأ ترجمة لأنه خاص.

ملاحظة جانبية: الكلمة

```
class
```

تُستعمل أيضًا في رأس القالب

```
template <class T>
```

ولا تصلح كلمة البنية مكانها، لكن هذا ليس فرقًا بين النوعين نفسيهما.

نصيحة للامتحان: ارسم جدولًا بسطرين (الرؤية الافتراضية للأعضاء، والوراثة الافتراضية)، ثم مثالًا قصيرًا لكل منهما، ثم جملة تقول إن باقي الفروق أسلوب وعُرف فقط.

Lecture link: 9/15 and 9/17 'C++ Objects' pages: "struct - primitive objects -> visibility = public" with struct name { string first, last; }; name n; n.first = "John"; and "class - full objects" (attributes + methods, visibility public/protected/private = information hiding, constructor/destructor, inheritance, polymorphism). Uses the instructor's wording 'primitive object' vs 'full object'. Template keyword note ties to 9/15 'template void bubbleT(...)'.

G14. When is it better to use a class

Answer

Use a **class** (private data + public methods) when the type is a "**full object**":

1. **It has rules (invariants) that must always be true**, e.g. an account balance is never negative, a Vector's `curr_element <= max_elements`, a valid student ID. Make the data **private** so it can only change through methods that check the rule (**information hiding**).
2. **It has behavior**: methods that work on the data (`print`, `compare`, `addAtEnd`, ...), not just storage.
3. **It manages a resource**, such as heap memory (`new/delete`) or files. It needs a constructor, a destructor, and a copy constructor/assignment that does a **deep copy** (lecture 9/22: a `Student` with `int *data`, where the default copy is shallow).
4. **The implementation may change later**: users see only the public interface, so the internals can change without changing user code.
5. **It is part of inheritance / polymorphism**, e.g. `Person -> Student` with `virtual print()`.

```
class Account {  
private:  
    double balance;           // rule: balance >= 0  
public:  
    Account() : balance(0) {}  
    void deposit(double amt) { if (amt > 0) balance += amt; }  
    bool withdraw(double amt) { // the ONLY way to reduce balance  
        if (amt <= 0 || amt > balance) return false;  
        balance -= amt;  
        return true;  
    }  
    double getBalance() const { return balance; }  
};
```

No outside code can write `a.balance = -500;`, because the class protects the rule (g++: `'double Account::balance'` is private within this context).

الشرح بالعربي خطوة بخطوة

السؤال هنا: متى نختار الصنف؟ أسأل نفسك هذه الأسئلة بالترتيب:

١. هل توجد قاعدة يجب أن تبقى صحيحة دائماً؟ مثل: الرصيد لا يصبح سالباً، أو عدد العناصر في المتجه لا يتجاوز السعة. إذا كان الجواب نعم فنحن نحتاج إخفاء المعلومات: نجعل البيانات خاصة، ولا تتغير إلا عبر طرق تتحقق من القاعدة. هذا هو عمل الصنف.

٢. هل للكائن سلوك؟ أي طرق تعمل على بياناته مثل الطباعة والمقارنة والإضافة، وليس مجرد تخزين. إذن صنف.

٣. هل يحجز الكائن ذاكرة من الكومة؟ عندها يحتاج مُنشئاً، ومُدمراً يحرر الذاكرة، ومُنشئ نسخ وعملية إسناد تعملان نسخاً عميقاً، لأن النسخ الافتراضي سطحي ويجعل كائنين يشيران إلى نفس الكتلة، كما رأينا في محاضرة النسخ. هذا صنف بالتأكيد.

٤. هل تريد أن تغيّر التنفيذ الداخلي لاحقًا دون أن يتأثر من يستخدم الكائن؟ الصنف يعرض الواجهة العامة فقط، فتغيير الداخل لا يكسر كود المستخدم.

٥. هل سيدخل في وراثة وتعدد أشكال؟ مثل الطالب الذي «هو» شخص، مع طريقة طباعة افتراضية. إذن صنف.

مثال: صنف الحساب البنكي، الرصيد فيه خاص، وطريقة السحب ترفض أي مبلغ أكبر من الرصيد، فلا يستطيع أحد من الخارج أن يجعل الرصيد سالبًا.

Lecture link: 9/17 'C++ Objects' (class = full objects: information hiding, constructor/destructor, inheritance, polymorphism; Person/Student); 9/22 copy page (class Student { private: int s_id; int *data; ... } -> default copy is shallow, write copy constructor Student(Student &) for deep copy); 9/22-9/24 Vector built as a class with arr2 / max_elements / curr_element.

G15. When is it better to use a struct

Answer

Use a **struct** when the type is a "**primitive object**", a plain group of related data:

1. It only **bundles values together** (a record / plain-old-data) and **any combination of values is valid**, so there is no rule (invariant) to protect and hiding the data gains nothing.
2. **All members are meant to be public** (struct is public by default, so less code).
3. **Little or no behavior**, at most a constructor or a tiny helper.
4. A small record returned from a function, or data passed to/from **C code** or C libraries. This works when the members are plain C types (int, double, char arrays, pointers), because the layout is then the same as a C struct.

Examples:

```
struct name { string first, last; };           // lecture 9/15
struct Point { double x, y; };
struct Node { int data; Node *next; };        // linked-list node

name n; n.first = "John";                     // direct access is fine
Point p = {3.0, 4.0};                         // brace (aggregate) initialization
```

Rule of thumb: if you can set any member to any value and the object is still valid, use a `struct`. If there is a rule to enforce or a resource (heap memory) to manage, use a `class` with private data.

الشرح بالعربي خطوة بخطوة

البنية مناسبة عندما يكون الكائن مجرد «صندوق» يجمع بيانات مترابطة، وأي قيم نضعها في أعضائه تُبقيه كائنًا صحيحًا.

الخطوة الأولى: انظر إلى البيانات. النقطة لها إحداثيان، وأي عددين يمثلان نقطة صحيحة. لا توجد قاعدة نحويها، إذن لا فائدة من إخفاء البيانات.

الخطوة الثانية: هل يوجد سلوك مهم؟ غالبًا لا، أو مجرد مُنشئ صغير.

الخطوة الثالثة: كل الأعضاء عامة، والبنية عامة تلقائيًا، فنكتب كودًا أقل وأوضح.

أمثلة: بنية الاسم من محاضرة الأستاذ (الاسم الأول والأخير)، والنقطة، وعقدة القائمة المرتبطة (قيمة ومؤشر إلى العقدة التالية)، وسجل صغير تعيده دالة، أو بيانات بسيطة (أعداد وأحرف ومؤشرات) نمررها إلى مكتبة مكتوبة بلغة

C

ميزة إضافية: يمكن تهيئة البنية بالأقواس المعقوفة مباشرة، مثل

```
Point p = {3.0, 4.0};
```

قاعدة سهلة للحفظ: إذا كان كل عضو يقبل أي قيمة ويبقى الكائن صحيحًا فاختر البنية. وإذا كانت هناك قاعدة يجب حمايتها أو ذاكرة يجب إدارتها فاختر الصنف ببيانات خاصة.

Lecture link: 9/15 'C++ Objects': "struct - primitive objects -> visibility = public"; instructor's example
struct name { string first, last; }; name n; n.first = "John";

G16. Explain the meaning of: Information Hiding, Inheritance, Polymorphism

Answer

Information hiding (closely tied to *encapsulation*, which means bundling data + methods in one unit): an object keeps its internal data and implementation details hidden and lets other code use it only through its **public interface** (methods). It is implemented with visibility `private` / `protected` / `public`. Benefits: data cannot be put into an invalid state, the implementation can change without breaking user code, less coupling. *Example:* Person's `ssn` is `private`, so only Person's own methods can read or change it.

Inheritance: creating a new class (**derived / child / subclass**) from an existing class (**base / parent / superclass**). The derived class automatically gets the base's attributes and methods, can **add** new ones, and can **override** (redefine) methods. It models an **"is-a"** relationship and gives code reuse. (The base's *private* members, such as `ssn`, are inherited too and exist inside every Student object, but Student's methods cannot access them directly.) *Example:* Student **is-a** Person: it inherits `firstname`, `lastname`, `write_name()`, adds `student_id`, and overrides `print()`. Student -> Person = *generalization*; Person -> Student = *specialization*. (In UML: hollow-triangle arrow pointing to the base; University: Person <- Student, Faculty, Admin, Guest. Student -mentor- Faculty is an *association*, not inheritance.)

Polymorphism ("many forms"): the same name/call does different things depending on the type of the object.

- *Compile-time (static):* **overloading**, e.g. `compare(int,int)` and `compare(float,float)`; the compiler picks the version from the argument types (templates are also static polymorphism).
- *Run-time (dynamic):* a **base-class pointer or reference** to a derived object calls the **derived** version of a **virtual** method, decided at run time from the real object type.

```
class Person {  
private:
```

```

    string ssn;                                // information hiding
protected:
    string firstname, lastname;
public:
    Person(string f, string l, string s) : ssn(s), firstname(f), lastname(l) {}
    virtual ~Person() {}
    virtual void print() const { cout << "Person: " << firstname << endl; }
};

class Student : public Person {                // inheritance: Student is-a Person
private:
    int student_id;
public:
    Student(string f, string l, string s, int id)
        : Person(f, l, s), student_id(id) {}
    void print() const override {               // overrides Person::print
        cout << "Student: " << firstname << " id=" << student_id << endl;
    }
};

int compare(int a, int b)    { return (a < b) ? -1 : (a > b); } // overloading
int compare(float a, float b) { return (a < b) ? -1 : (a > b); } // (compile-time)

int main() {
    Student *s = new Student("John", "Doe", "123-45-6789", 42);
    s->print();           // Student version
    Person *p = s;        // base pointer -> Student object
    p->print();            // Student version (run-time polymorphism, needs virtual)
    delete p;             // virtual destructor -> ~Student() also runs
    return 0;
}

```

Worked steps / details

Output:

```

Student: John id=42
Student: John id=42

```

Why virtual matters: if `virtual` is removed from `Person::print`, then `p->print()` prints `Person: John`. Without `virtual` the call is bound at **compile time** from the **pointer type** (`Person*`); with `virtual` it is bound at **run time** from the **object type** (`Student`).

Pointer or reference only (no slicing): `Person &r = *s; r.print();` also prints the `Student` version. But `Person sliced = *s; sliced.print();` prints `Person: John` even with `virtual`, because copying into a plain `Person` object *slices off* the `Student` part.

`compare(3, 5)` calls the `int` version and returns `-1`; `compare(2.5f, 1.5f)` calls the `float` version and returns `1`. The compiler chooses by argument types (overloading).

الشرح بالعربي خطوة بخطوة

هذه ثلاث أفكار هي أعمدة البرمجة الكائنية. سنشرحها على مثال الأستاذ: الشخص والطالب.

أولاً، إخفاء المعلومات: الكائن يُخفي بياناته الداخلية، ويعرض للعالم الخارجي واجهة فقط، وهي الطرق العامة. في مثال الشخص، رقم الضمان الاجتماعي خاص، فلا تستطيع الدالة الرئيسية أن تقرأه أو تغيره مباشرة، بل عبر طرق الصنف فقط.

الفائدة: حماية البيانات من القيم الخاطئة، وإمكانية تغيير التنفيذ الداخلي دون كسر الكود الذي يستخدم الكائن.

ثانيًا، الوراثة: نبنى صنفًا جديدًا من صنف موجود. الطالب «هو» شخص، فيرث منه الاسم الأول والأخير وطريقة كتابة الاسم، ثم يضيف رقم الطالب، ويعيد كتابة طريقة الطباعة. الاتجاه من الطالب إلى الشخص يسمى تعميمًا، ومن الشخص إلى الطالب يسمى تخصيصًا. في مخططات

UML

نرسم سهمًا برأس مثلث فارغ نحو الصنف الأب. انتبه: علاقة المرشد بين الطالب وعضو هيئة التدريس ليست وراثة بل ارتباط. ملاحظة: الأعضاء الخاصة في الشخص، مثل رقم الضمان، تُورث أيضًا، أي أنها موجودة داخل كائن الطالب، لكن طرق الطالب لا تستطيع الوصول إليها مباشرة.

ثالثًا، تعدد الأشكال: نفس الاستدعاء يتصرف بشكل مختلف حسب نوع الكائن الحقيقي. وله نوعان:

- وقت الترجمة: التحميل الزائد، أي دالتان بنفس الاسم ومعاملات مختلفة، مثل دالة المقارنة للأعداد الصحيحة ودالة المقارنة للأعداد العشرية، والمترجم يختار حسب نوع المعاملات.
- وقت التشغيل: مؤشر أو مرجع من نوع الشخص يشير إلى كائن طالب، فإذا استدعينا الطباعة تُنفَّذ نسخة الطالب، بشرط أن تكون الطباعة في صنف الشخص معلنة بالكلمة

virtual

وبدون هذه الكلمة تُنفَّذ نسخة الشخص، لأن القرار يُؤخذ من نوع المؤشر لا من نوع الكائن.

انتبه أيضًا: لو نسخنا الطالب في متغير عادي من نوع الشخص، وليس في مؤشر أو مرجع، فإن الجزء الخاص بالطالب يُقطع (تقطيع الكائن)، وتُنفَّذ نسخة الشخص حتى مع وجود

virtual

للامتحان: عرّف كل مصطلح في سطرين، ثم اكتب المثال القصير، وأشر إلى السطر

p->print()

الذي يطبع نسخة الطالب.

Lecture link: 9/17 'C++ Objects' page: Person (firstname, lastname, ssn (private); write_name, print(), compare()) and Student is-a Person (generalization / specialization, student_id, print(), compare(s)); Student *s = new Student; s->print() // Student version; Person *p = s; p->print() // Student version; C++ overloading compare(int,int) / compare(float,float). 9/17 UML University Personnel (inheritance arrows vs 'mentor' association). Instructor's student.h also uses a virtual destructor.

G17. What is the meant by private and public visibility

Answer

Visibility (access control) decides *which code is allowed to use* (read, write, or call) a member of a class. It is the mechanism that implements **information hiding**.

- **public:** accessible by **any** code (other functions, `main`, other classes). The public members form the object's **interface**, e.g. `s1.print(cout)`, `s1.set(123, "John Doe")`.
- **private:** accessible **only inside the class's own member functions** (and `friends`). Code outside the class (`main`, other classes, even derived classes) cannot use it. Usually the attributes: `s_id`, `name`, `ssn`.
- **(protected:** like private for outside code, but **also** accessible by derived classes.)

Who is accessing	public	protected	private
member functions of the same class (and friends)	yes	yes	yes
member functions of a derived class	yes	yes	no
any other code (e.g. <code>main</code>)	yes	no	no

Notes:

- Access is checked **at compile time**: a violation is a compile error (no run-time cost).
- Access is **per class, not per object**: a `Student` method may use the private members of *another* `Student` object (e.g. in a copy constructor `Student(const Student &o) : s_id(o.s_id), name(o.name) {}`).
- Purpose: outside code cannot put the object into an invalid state; it must go through public methods that can check values.

Worked steps / details

Instructor's `student.h`, annotated:

```
class Student {
private:
    int s_id;                // only Student's methods can use
    std::string name;
public:
    Student();               // any code can use
    virtual ~Student();
    void set(int, std::string);
    std::ostream& print(std::ostream&);
};

// in main:
Student s1;
s1.set(123, "John Doe");    // OK - public
s1.print(cout);             // OK - public
// s1.s_id = 5;             // ERROR - s_id is private
```

g++ for the last line: error: 'int Student::s_id' is private within this context.

الشرح بالعربي خطوة بخطوة

الرؤية تعني: من المسموح له أن يلمس هذا العضو، أي يقرأ المتغير أو يغيّره أو يستدعي الطريقة؟

تخيّل الصنف كبيت:

• العام

public

هو الباب الأمامي: أي كود في البرنامج يستطيع استخدامه، مثل طريقة الطباعة وطريقة الضبط في مثال الطالب. هذه هي واجهة الكائن.

• الخاص

private

غرفة مقفلة لا يدخلها إلا طرق الصنف نفسه (والأصدقاء). مثل رقم الطالب والاسم في صنف الطالب. حتى الصنف الابن لا يدخلها.

• المحمي

protected

غرفة مقفلة عن الغرباء لكنها مفتوحة لأبناء العائلة، أي الأصناف الموروثة.

ملاحظة مهمة: الحماية على مستوى الصنف لا على مستوى الكائن، فطريقة في صنف الطالب تستطيع قراءة البيانات الخاصة لكائن طالب آخر، كما في مُنشئ النسخ.

ملاحظة ثانية: المخالفة تُكتشف وقت الترجمة، أي أن البرنامج لن يُترجم أصلاً، فهي ليست خطأ وقت التشغيل.

لماذا نفعل هذا؟ لتطبيق إخفاء المعلومات: البيانات مخفية، والتعامل معها يمر عبر طرق عامة تتحقق من صحة القيم، فلا يستطيع أحد أن يضع قيمة خاطئة.

Lecture link: 9/15-9/17 'C++ Objects': "attributes, methods -> visibility (public, protected, private) (information hiding)"; Person with ssn (private); instructor's student.h (private s_id, name; public Student(), ~Student(), set(), print()) and main.cpp (s1.set(123,"John Doe"); s1.print(cout);).

G18. How are private and public visibility applied in C++

Answer

1. **Access-specifier labels** inside the class declaration: `public:`, `private:`, `protected:`. A label applies to **every member after it until the next label**; labels can appear in any order and more than once.
2. **Defaults** when no label is written: `class` -> `private`, `struct` -> `public`.
3. **Usual pattern:** data members `private`; constructors, getters/setters and other methods `public`; internal helper methods `private` (e.g. a `Vector`'s `resize()`). A setter can **validate** a value before storing it.
4. Using a private member from outside the class is a **compile-time error**.
5. **Inheritance access** is written too: `class Student : public Person` means `Person`'s public members stay public in `Student` (the normal "is-a"). With `: protected` / `: private` they become protected / private in the derived class; the default for `class` is `private`. The base's **private** members are inherited but never directly accessible in the derived class (only through the base's public/protected methods).
6. A `friend` function/class may access private members (the exception).

```

class Account {
private:                                // everything below is private ...
    double balance;
    int pin;
public:                                // ... until this label
    Account() : balance(0.0), pin(0) {}
    void deposit(double amt) { if (amt > 0) balance += amt; } // setter with check
    double getBalance() const { return balance; }           // getter
protected:
    void setPin(int p) { pin = p; }
};

class Savings : public Account {
public:
    void init() { setPin(1234); } // OK: protected is accessible in derived class
    // void bad() { balance = 5; } // ERROR: base's private is not accessible
};

int main() {
    Account a;
    a.deposit(100);                // OK - public
    cout << a.getBalance();        // OK - public
    a.balance = 1000000;          // COMPILER ERROR - balance is private
    return 0;
}

```

Worked steps / details

g++ output for the line `a.balance = 1000000;`:

```

error: 'double Account::balance' is private within this context
    a.balance = 1000000;
    ^~~~~~
note: declared private here
    double balance;

```

Without that line the program compiles and prints `100`. (If `bad()` is uncommented: the same "is private within this context" error inside `Savings`. Calling `a.setPin(3)` from `main` gives: 'void Account::setPin(int)' is protected within this context.)

الشرح بالعربي خطوة بخطوة

كيف نطبق ذلك عملياً في

C++

؟ خطوة بخطوة:

١. داخل إعلان الصنف نكتب علامة الوصول متبوعة بنقطتين، وهي واحدة من ثلاث:

private:

أو

public:

أو

protected:

٢. كل عضو يأتي بعد العلامة يأخذ نفس الرؤية حتى تظهر علامة جديدة. ويمكن تكرار العلامات وبأي ترتيب.

٣. إذا لم تكتب أي علامة: الصنف خاص تلقائيًا، والبنية عامة تلقائيًا.

٤. النمط المعتاد: المتغيرات خاصة، والمُنشئ وطرق القراءة والضبط وباقي الطرق عامة، والطرق المساعدة الداخلية خاصة، مثل طريقة تكبير المصفوفة داخل المتجه.

٥. طريقة الضبط تستطيع أن تتحقق من القيمة قبل تخزينها، مثل رفض الإيداع السالب، وهذه هي فائدة الإخفاء.

٦. لو كتبت في الدالة الرئيسية سطرًا يغيّر الرصيد الخاص مباشرة، يرفض المترجم ويقول إن العضو خاص في هذا السياق.

٧. في الوراثة نكتب الرؤية أيضًا، مثل

```
class Student : public Person
```

ومعناها أن الأعضاء العامة في الشخص تبقى عامة في الطالب، وهذا المطلوب لعلاقة «هو». لو نسيت كلمة العام في الصنف تصبح الوراثة خاصة تلقائيًا. والأعضاء الخاصة في الأب موجودة داخل كائن الابن، لكن طرق الابن لا تستطيع الوصول إليها مباشرة أبدًا، أما المحمية فتستطيع.

٨. استثناء: الدالة أو الصنف المعلن صديقًا بالكلمة

friend

يستطيع الوصول إلى الخاص.

Lecture link: 9/17: class Student { private: int student_id; public: void print(); }; written in student.h; instructor's student.h / student.cpp / main.cpp use private data + public set()/print(). 9/17 Student is-a Person -> class Student : public Person. Error text verified with g++ 13.3 (-Wall -Wextra -std=c++11).

G19. What is the difference between the declaration and definition of a class

Answer

Class declaration (in the header file, e.g. `student.h`): describes **what** the class is: its name, its data members and their types, the member-function **prototypes** (name, parameters, return type), and their visibility. No function bodies (except small inline ones). It tells the compiler the **size/layout** of an object and lets it **check every call**. It is `#include`d in every `.cpp` that uses the class.

Class definition (in the source file, e.g. `student.cpp`): the **actual code** (bodies) of the member functions, written outside the class with the scope operator `Student::` ("this `print` belongs to class `Student`"). Inside a method, `this` points to the calling object. It is compiled once into `student.o`, and the linker connects it to the calls.

```
// student.h --- DECLARATION
#ifndef STUDENT_H_
#define STUDENT_H_
class Student {
private:
```

```

    int student_id;
public:
    Student();
    void setID(int id);
    void print() const;
};
#endif

```

```

// student.cpp --- DEFINITION
#include <iostream>
#include "student.h"
Student::Student() : student_id(-1) {}
void Student::setID(int id) { this->student_id = id; }
void Student::print() const {
    std::cout << "Student ID: " << student_id << std::endl;
}

```

```

// main.cpp --- USE
#include "student.h"
int main() { Student s; s.setID(123); s.print(); }

```

Build: `g++ -c student.cpp, g++ -c main.cpp, g++ student.o main.o -o student.`

- Missing declaration -> **compile** error ('Student' was not declared in this scope).
- Missing definition -> compiles, but **link** error: undefined reference to 'Student::Student()'.

Formal note: `class Student;` alone is a (forward) **declaration**: an incomplete type, so only pointers/references are allowed. The body `class Student { ... };` is formally the *class definition*, and the `Student::` bodies are the *member-function definitions*. **Templates** are the exception to the .h/.cpp split: the whole template goes in the header, because the compiler needs the source to generate code for each type.

Worked steps / details

Forward declaration only:

```

class Course;           // declaration only (incomplete type)
Course *cp = nullptr;   // OK - pointer size is known
// Course c;           // ERROR - size of Course unknown

```

(g++: aggregate 'Course c' has incomplete type and cannot be defined.)

The header guard (`#ifndef STUDENT_H_ / #define / #endif`) stops the class body from being compiled twice when the header is included more than once. Without it g++ reports `error: redefinition of 'class Student'`, which also shows that the body is formally a definition.

الشرح بالعربي خطوة بخطوة

الفكرة: نقسم الصنف إلى جزأين في ملفين.

الجزء الأول، الإعلان، في ملف الترويسة

`student.h`

هنا نصف الصنف: اسمه، ومتغيراته وأنواعها، ورؤوس الطرق (الاسم والمعاملات ونوع الإرجاع)، مع الرؤية. لا توجد أجسام للطرق. هذا يكفي المترجم ليعرف حجم الكائن، ولتحقق أن كل استدعاء صحيح. لذلك كل ملف يستخدم صنف الطالب يضمن هذه الترويسة.

الجزء الثاني، التعريف، في ملف المصدر

`student.cpp`

هنا نكتب الكود الفعلي لكل طريقة، ونضع قبل اسمها اسم الصنف ثم نقطتين مزدوجتين، مثل

`Student::print`

لنخبر المترجم أن هذه الطباعة تابعة لصنف الطالب، أي في نطاقه. وداخل الطريقة يوجد المؤشر

`this`

الذي يشير إلى الكائن الذي استدعاه.

ماذا يحدث لو نقص جزء؟ بدون الإعلان: خطأ ترجمة لأن الصنف غير معروف. بدون التعريف: الترجمة تنجح لكن الربط يفشل برسالة «مرجع غير معرف».

ملاحظة من المصطلحات الرسمية: السطر

`class Student;`

وحده إعلان مسبق، وبه نستطيع فقط صنع مؤشر أو مرجع. أما جسم الصنف بين القوسين فيسمى رسميًا تعريف الصنف. لكن في محاضرة الأستاذ نسمي ملف الترويسة إعلانًا وملف المصدر تعريفًا، فكتب ذلك في الامتحان مع الملاحظة.

استثناء القوالب: كود القالب كله يوضع في الترويسة، لأن المترجم يحتاج المصدر ليولد نسخة لكل نوع.

ولا تنس حُرّاس الترويسة، فهي تمنع تكرار جسم الصنف إذا ضُمن الملف مرتين، وبدونها يشتكي المترجم من إعادة تعريف الصنف.

Lecture link: 9/17 right side: "class -> declaration / definition"; `class Student { private: int student_id; public: void print(); };` in `student.h`, and `#include "student.h" void Student::print() { ... }` in `student.cpp`, with 'Student::' labeled 'namespace (scope)' and 'this ptr'; instructor's `student.h / student.cpp / main.cpp`; 9/15 templates: compiler needs the source -> template code goes in the `.h`. Instructor's convention: declaration = `.h`, definition = `.cpp`.

G20. What are the main properties of a Vector

Answer

A **Vector** is a **self-sizing (dynamic) array**:

1. **Contiguous storage on the heap:** the elements are in an array allocated with `new (arr2)`; the vector object holds the pointer.
2. **Random access in O(1):** get or change the value at any index directly (`arr2[i]`).

3. **Size vs capacity:** `curr_element` = number of elements used (size) $\leq$ `max_elements` = number of slots allocated (capacity).
4. **Grows automatically:** `addAtEnd(e)` : check if full; if full -> **resize** (allocate a bigger array, copy the old elements, delete `[]` the old array, update `max_elements`); then `arr2[curr_element] = e;`
`curr_element++;`
5. **Growth policy:** +1, +constant (e.g. +100), or x factor (e.g. x2). With +1 or +constant the total copying for N adds is $O(N^2)$, i.e. $O(N)$ per add on average. With **doubling** it is $O(N)$ total, i.e. **amortized $O(1)$** per `addAtEnd`. (With a factor, the capacity must start at 1 or more: 2×0 is still 0.)
6. **Add/remove at the end: $O(1)$** (amortized). **Add/remove at the start or middle: $O(N)$** , because elements must be shifted right/left.
7. **Can shrink / compress** (resize down to `curr_element`) to free unused memory.
8. **Owns heap memory**, so it is written as a class with private data, a destructor (`delete []`), and a deep-copy copy constructor/assignment (the default copy is shallow, so two vectors would share one array).
9. **Pitfalls:** forgetting `delete [] temp` in `resize` -> **memory leak**; using an old pointer to the elements after a `resize` -> **use after deallocation** (dangling pointer).
10. **Generic:** works for any element type: `std::vector<T>` in C++ (`push_back`, `pop_back`, `insert`, `erase`, `size()`, `capacity()`, `reserve()`, `shrink_to_fit()`, `operator[]` with no bounds check, `at()` with bounds check that throws `std::out_of_range`).

Operation	Time
get / change value at index	$O(1)$
add at end	$O(1)$ amortized ($O(N)$ when a resize happens)
remove at end	$O(1)$
add / remove at start or middle	$O(N)$ (shift)
resize	$O(N)$ (copy)

```

void IntVec::addAtEnd(const int &e) {
    if (curr_element == max_elements)           // full?
        resize(max_elements > 0 ? 2 * max_elements : 1); // true: resize (x2)
    arr2[curr_element] = e;                     // add to next location
    curr_element++;
}

void IntVec::resize(int newMax) {
    int *temp = arr2;                          // keep old block
    arr2 = new int[newMax];                    // bigger block on heap
    for (int i = 0; i < curr_element; i++)     // copy data
        arr2[i] = temp[i];
    delete [] temp;                           // missing -> MEMORY LEAK
    max_elements = newMax;
}

```

Worked steps / details

Trace: start with capacity 2, growth x2, add 10, 20, 30, 40, 50:

add	size (curr_element)	capacity (max_elements)	resize?
10	1	2	no
20	2	2	no
30	3	4	yes (copy 2)
40	4	4	no
50	5	8	yes (copy 4)

Why doubling gives amortized $O(1)$: for N adds starting at capacity 1, the copies are $1 + 2 + 4 + \dots + N/2 < N$, so the total work is $O(N)$ and the average per add is $O(1)$. **With +1:** copies = $1 + 2 + \dots + (N-1) = N(N-1)/2$, so $O(N^2)$ total and $O(N)$ per add. **With +100:** about $N^2/200$ copies, still $O(N^2)$, but 100 times fewer resizes.

std::vector usage:

```
vector<int> sv;
sv.push_back(5); sv.push_back(3); sv.push_back(8); // add at end
sv.insert(sv.begin(), 1); // add at start: O(N) shift -> 1 5 3 8
sv.pop_back(); // remove at end: O(1) -> 1 5 3
sv[1] = 7; // change value, no bounds check -> 1 7 3
try {
    sv.at(10) = 1; // bounds check -> throws std::out_of_range
} catch (const std::out_of_range &e) {
    cout << "index 10 is out of range" << endl;
}
```

الشرح بالعربي خطوة بخطوة

المتجه هو مصفوفة تغيّر حجمها بنفسها. المصفوفة العادية حجمها ثابت ولا تكبر، والمتجه يحل هذه المشكلة. لنفهم خصائصه خطوة بخطوة:

١. التخزين: العناصر متجاورة في مصفوفة على الكومة، والمتجه يحمل مؤشرًا إليها. لذلك الوصول إلى أي عنصر برقم موقعه سريع جدًا وثابت الزمن

$O(1)$

سواء للقراءة أو للتغيير.

٢. رقمان مهمان: الحجم وهو عدد العناصر المستخدمة فعليًا، والسعة وهي عدد الأماكن المحجوزة. الحجم دائمًا أقل من السعة أو يساويها.

٣. الإضافة في النهاية: نسال: هل المصفوفة ممتلئة؟ إن لم تكن، نضع العنصر في المكان التالي ونزيد العداد. وإن كانت ممتلئة، نكبّرها أولًا.

٤. التكبير: نحجز مصفوفة أكبر، ثم ننسخ العناصر القديمة، ثم نحرر المصفوفة القديمة، ثم نحدّث السعة. إذا نسيت تحرير القديمة يحدث تسريب ذاكرة، وإذا استعملت المؤشر القديم بعد تحريره فهذا استخدام بعد التحرير.

٥. مقدار التكبير: زيادة واحد (نسخ في كل إضافة، بطيء جدًا)، أو زيادة ثابتة مثل مئة، أو الضرب في معامل مثل الضعف. مع المضاعفة يصبح متوسط كلفة الإضافة في النهاية ثابتًا، لأن مجموع كل عمليات النسخ أقل من عدد العناصر. أما مع زيادة واحد فالمجموع يصبح

$$O(N^2)$$

وكذلك مع الزيادة الثابتة مثل مئة: هي تقلل عدد مرات التكبير فقط، لكن رتبة المجموع تبقى تربيعية. انتبه أيضًا: إذا بدأت السعة من صفر فالضعف يبقى صفرًا، لذلك نبدأ بسعة واحد على الأقل.

٦. الإضافة أو الحذف في البداية أو الوسط أبطأ

$$O(N)$$

لأننا نزيح العناصر يمينًا أو يسارًا لنفتح مكانًا أو نسد فراغًا.

٧. يمكن تصغيره أو ضغطه ليساوي عدد العناصر فنوفر الذاكرة.

٨. لأنه يملك ذاكرة على الكومة، نكتبه صنفًا ببيانات خاصة ومدمر ومُنشئ نسخ عميق، وإلا سيشارك متجهان في نفس المصفوفة بسبب النسخ السطحي.

٩. في المكتبة القياسية يوجد

`std::vector<T>`

ويعمل مع أي نوع، وفيه دالة الإضافة في النهاية ودالة الحجم ودالة السعة، والوصول بالأقواس المربعة بدون فحص حدود، أو بالدالة

`at()`

مع فحص الحدود، وهي ترمي استثناء إذا كان الموقع خارج الحدود.

Lecture link: 9/22 'Self-sizing Array - Vector' (resize inc/dec, compress; add/remove at start/mid/end; change values; get values at loc; `int *arr2 = new int[100]; int *temp = arr2; arr2 = new int[200]; copy data from temp to arr2; delete temp[]` // if missing -> memory leak; 'use after de-allocation'). 9/24 Vector: `addAtEnd(elem &e)`: check if full -> true: resize, false: `arr2[curr] = e; curr++;` fields `arr2`, `max_elements = N`, `curr_element`; resize increases `max_elements` by +1, `+(factor)(max_elements)`, or +100. Note: the lecture writes '`delete temp[]`'; the correct C++ syntax is '`delete [] temp`'.

Part 2 — Sample1: Long Questions

L1. Explain the operation of Mergesort, in particular how is the data split, how merging operates, why the time complexity is $O(N \lg N)$, and what happens if the data is in complete reverse order as to what is required (want ascending order and data is in descending order).

Answer

Idea: divide and conquer. Split the array into halves until each piece has one element (a 1-element array is already sorted). Then merge the sorted pieces back together.

1. How the data is split

```
void merge_sort(int A[], int work[], int start, int end) {
    if (start >= end) return;           // 0 or 1 element: already sorted
    int mid = (start + end) / 2;        // split point
    merge_sort(A, work, start, mid);    // sort left half A[start..mid]
    merge_sort(A, work, mid + 1, end);  // sort right half A[mid+1..end]
    merge(A, work, start, mid, end);    // merge the two sorted halves
}
```

- The split is by **position only** (the middle index). No comparisons are made and no data moves.
- The recursion continues until pieces have size 1: N , then $N/2$, $N/4$, ..., 1.

2. How merging works (two sorted runs, indices i, j, k)

```
void merge(int A[], int work[], int start, int mid, int end) {
    int i = start, j = mid + 1, k = start;
    while (i <= mid && j <= end) {      // both runs still have elements
        if (A[i] <= A[j]) work[k++] = A[i++]; // take the smaller one
        else work[k++] = A[j++];
    }
    while (i <= mid) work[k++] = A[i++]; // copy the rest of the left run
    while (j <= end) work[k++] = A[j++]; // copy the rest of the right run
    for (k = start; k <= end; k++) A[k] = work[k]; // copy back
}
```

- Compare the front elements of the two runs, copy the smaller one into the work array, and advance that index. When one run is used up, copy the rest of the other run, which is already sorted.
- Every element is copied once into the work array (and once back), so merging n elements costs $O(n)$ with at most $n-1$ comparisons. This needs an $O(N)$ **work array**.
- Lecture example: $A = 1\ 9\ 12\ 14$ and $B = 3\ 6\ 10$ merge into $C = 1\ 3\ 6\ 9\ 10\ 12\ 14$ (5 comparisons, then 12 and 14 are copied because B is done).

3. Why the time is $O(N \log_2 N)$

- **Number of levels:** halving N until the size is 1 takes $\log_2 N$ levels. For $N = 8$ that is 3 levels.
- **Work per level:** at every level the pieces together hold all N elements, and each element is copied once while merging. That is about N work per level.
- **Total:** $N \times \log_2 N$. As a recurrence: $T(N) = 2T(N/2) + cN$, so $T(N) = cN \log_2 N + cN = O(N \log_2 N)$.

- The split never depends on the data, so best = average = worst = $O(N \log_2 N)$.

4. Data in complete reverse order (we want ascending, the data is descending)

- The splitting is exactly the same: the same pieces and the same $\log_2 N$ levels.
- In every merge, **every element of the right run is smaller than every element of the left run**. So the whole right run is copied first, one comparison per element, and then the left run is copied with no comparisons.
- So each merge of n elements needs only **$n/2$ comparisons** (the size of the right run), fewer than for random data. The number of copies is the same (N per level).
- **Result: still $O(N \log_2 N)$** . Reverse order is **not** a worst case for mergesort. Compare insertion sort, which needs $N(N-1)/2 = 28$ comparisons and 28 shifts for $N = 8$, and quicksort with pivot = $A[\text{end}]$, which becomes $O(N^2)$. Mergesort is not sensitive to the input order, but it also gains nothing from it.

Worked steps / details

Worked example: 8 7 6 5 4 3 2 1 (checked with a program)

Split phase ($\log_2 8 = 3$ levels):

```
[8 7 6 5 4 3 2 1]
[8 7 6 5]          [4 3 2 1]
[8 7] [6 5]        [4 3] [2 1]
[8] [7] [6] [5]    [4] [3] [2] [1]
```

Merge phase (copies = copies into the work array):

Level	Merges	Result	Comparisons	Copies
1 (size 1+1)	[8]+[7], [6]+[5], [4]+[3], [2]+[1]	[7 8] [5 6] [3 4] [1 2]	$4 \times 1 = 4$	8
2 (size 2+2)	[7 8]+[5 6], [3 4]+[1 2]	[5 6 7 8] [1 2 3 4]	$2 \times 2 = 4$	8
3 (size 4+4)	[5 6 7 8]+[1 2 3 4]	[1 2 3 4 5 6 7 8]	4	8
Total			12 = $(N/2) \times \log_2 N$	24 = $N \times \log_2 N$

The last merge in detail: 5 vs 1, copy 1. 5 vs 2, copy 2. 5 vs 3, copy 3. 5 vs 4, copy 4. The right run is now empty, so 5 6 7 8 are copied with no comparisons.

Other orders for comparison:

- $N = 8$: already sorted 1..8 takes 12 comparisons and 24 copies. The interleaved order 5 1 7 3 8 2 6 4 takes 17 comparisons (the maximum for $N = 8$) and 24 copies.
- $N = 1024$: reverse order takes 5,120 comparisons (= 512×10) and a random order takes about 8,945. Both make 10,240 copies (= 1024×10).

Lecture example 5 4 8 1 3 6: split into [5 4 8] [1 3 6], then [5 4] [8] and [1 3] [6]. Merge [4 5] and [1 3], then [4 5 8] and [1 3 6], then [1 3 4 5 6 8].

الشرح بالعربي خطوة بخطوة

الترتيب بالدمج يعتمد على مبدأ «فَرَق تَسُد»: نقسم المشكلة الكبيرة إلى مشاكل صغيرة سهلة، ثم نجمع حلولها.

أولاً: كيف نقسم البيانات؟

نحسب منتصف الجزء فنحصل على نصف أيسر ونصف أيمن. ثم نكرر الشيء نفسه على كل نصف (استدعاء ذاتي) حتى يصبح كل جزء عنصراً واحداً، والعنصر الواحد مرتب بطبيعته. انتبه: التقسيم يعتمد على المواقع فقط، فلا توجد في هذه المرحلة مقارنات ولا نقل للبيانات.

ثانياً: كيف يعمل الدمج؟

عندنا جزءان مرتبان. نضع مؤشراً على أول عنصر في كل جزء، ومؤشراً ثالثاً على أول خانة في مصفوفة العمل. نقارن العنصرين وننسخ الأصغر إلى مصفوفة العمل، ثم نحرك مؤشره ومؤشر مصفوفة العمل خطوة واحدة. نكرر ذلك حتى ينتهي أحد الجزأين، ثم ننسخ بقية الجزء الآخر كما هي لأنها مرتبة أصلاً. مثال المحاضرة: دمج

1 9 12 14

مع

3 6 10

يعطي

1 3 6 9 10 12 14

كل عنصر يُنسخ مرة واحدة إلى مصفوفة العمل (ثم يرجع مرة إلى المصفوفة الأصلية)، لذلك كلفة الدمج تتناسب مع عدد العناصر المدموجة.

ثالثاً: لماذا الزمن $n \times \log_2 n$ ؟

• عدد المستويات: نقسم n على 2 مرة بعد مرة حتى نصل إلى 1، وهذا يحتاج $\log_2 n$ مستوى. مثلاً ثمانية عناصر تصبح 4 ثم 2 ثم 1، أي 3 مستويات.

• العمل في كل مستوى: الأجزاء في أي مستوى تحتوي معاً كل العناصر، وكل عنصر يُنسخ مرة واحدة في الدمج، إذن عمل المستوى الواحد = n .

• المجموع = $n \times \log_2 n$. ولأن التقسيم لا يعتمد على القيم، فالزمن نفسه في أفضل حالة وفي أسوأ حالة.

رابعاً: ماذا لو كانت البيانات معكوسة تماماً؟

نريد ترتيباً تصاعدياً والبيانات تنازلية، مثل:

8 7 6 5 4 3 2 1

- التقسيم هو نفسه تماماً: نفس الأجزاء ونفس عدد المستويات (3).
- في كل دمج تكون كل عناصر النصف الأيمن أصغر من كل عناصر النصف الأيسر. مثلاً في الدمج الأخير بين الجزء الأيسر (من 5 إلى 8) والجزء الأيمن (من 1 إلى 4): نقارن 5 مع 1 فننسخ 1، ثم 5 مع 2، ثم 5 مع 3، ثم 5 مع 4. عندها ينتهي الجزء الأيمن، فننسخ الأيسر كله بدون أي مقارنة.
- إذن عدد المقارنات في كل دمج يساوي نصف العناصر فقط، وهذا أقل مما تحتاجه البيانات العشوائية، أما عدد النسخ فهو نفسه. تحققنا بالبرنامج: ثمانية عناصر معكوسة احتاجت 12 مقارنة و24 نسخة، وترتيب متداخل احتاج 17 مقارنة و24 نسخة.
- النتيجة: الزمن يبقى $N \times \log_2 N$ ، والبيانات المعكوسة ليست أسوأ حالة للدمج. هذا بعكس الإدراج الذي يحتاج 28 مقارنة و28 إزاحة لثمانية عناصر معكوسة، وبالعكس الترتيب السريع بمحور آخر عنصر الذي يصبح رتبة N^2 . أي أن الدمج لا يتأثر بترتيب البيانات.

Lecture link: 9/1 merge algorithm page: A = 1 9 12 14 and B = 3 6 10 merged into C with indices i, j, k, then copy the rest of A and the rest of B ('6. j is done'). 9/1 mergesort diagram: recursive split of 5 4 8 1 3 6, merges marked M, 'Array of size N => log2 N # of divisions', N work per level so $O(N \log_2 N)$, merge_sort(A, start, end) with C, D, E, and 'A, Work' for the work array. Two conventions differ from the notes. First, $mid = (start+end)/2$ is used here because it reproduces the lecture's split [5 4 8][1 3 6]; the page's formula $(end-start+1)/2 + start$ gives $mid = 3$, i.e. [5 4 8 1][3 6]. Second, the lecture's merge compares with $A[i] < B[j]$; using $\leq$ keeps equal keys in their original order (stable). Both versions sort correctly.

L2. For the given array, explain and show the results at the various stages of Selection sort as the array is sorted in ascending order. Explain why, even both methods are $O(N^2)$, there may be a difference in actual runtimes for Selection Sort and Insertion sort depending on the data in the array. Array: 10 2 12 5 3

Answer

Selection sort, lecture version: in each pass, find the **MAX** of the unsorted part and swap it into the **LAST unsorted position** $a[N-1-i]$. The sorted part grows from the right.

Array (N = 5): 10 2 12 5 3

Pass	Unsorted part searched	Max (index)	Swap with	Array after the pass	Comparisons	Swaps
start	–	–	–	10 2 12 5 3	–	–
1	10 2 12 5 3	12 (index 2)	a[4] = 3	10 2 3 5 12	4	1
2	10 2 3 5	10 (index 0)	a[3] = 5	5 2 3 10 12	3	1
3	5 2 3	5 (index 0)	a[2] = 3	3 2 5 10 12	2	1
4	3 2	3 (index 0)	a[1] = 2	2 3 5 10 12	1	1
5	2	– (one element)	–	2 3 5 10 12	0	0
Total					10 = N(N-1)/2	4

```

void selectionSort(int a[], int N) {
    for (int i = 0; i < N - 1; i++) {           // pass i+1 (lecture: i < N; the last pass has 1
        element)
            int max_loc = 0;
            for (int j = 1; j < N - i; j++)      // search a[0..N-1-i]
                if (a[j] > a[max_loc]) max_loc = j;
            if (max_loc != N - 1 - i)           // max already in place -> 0 swaps (lecture P4)
                swap(a[max_loc], a[N - 1 - i]); // max to the end of the unsorted part
    }
}

```

The textbook MIN-to-front version is also accepted (see the steps). It also takes 10 comparisons and 4 swaps.

Why selection and insertion sort can take different actual times although both are $O(N^2)$

- Big-O only describes the growth rate. It hides constant factors and ignores how the work depends on the data.
- **Selection sort does not depend on the data.** It always makes $N(N-1)/2$ comparisons (10 here), whether the data is sorted, random or reversed. It makes at most $N-1$ swaps, so it writes little.
- **Insertion sort depends on the data.** Its work is about $N + (\text{number of inversions})$:
 - sorted: $N-1$ comparisons and 0 shifts, which is $O(N)$
 - random: about $N^2/4$
 - reverse: $N(N-1)/2$ comparisons **and** $N(N-1)/2$ shifts
- **For this array** (it has 6 inversions): insertion sort makes **9 comparisons and 6 shifts**, and selection sort makes **10 comparisons and 4 swaps**. Here they do almost the same work.
- The same 5 values in other orders:
 - sorted (2 3 5 10 12): selection 10 comparisons and 0 swaps; insertion 4 comparisons and 0 shifts, so insertion is much faster.
 - reversed (12 10 5 3 2): selection 10 comparisons and 2 swaps; insertion 10 comparisons and 10 shifts, so insertion does many more writes.
- **Cost of a swap versus a comparison:** a swap is 3 assignments and a shift is 1. When elements are large (records or objects), moving them dominates the cost, so selection's few swaps win on random and reverse data. On nearly-sorted data insertion wins by far.
- **Cache and hardware:** both scan contiguous memory. Insertion works near elements it has just touched and exits its loop early, and on nearly-sorted data its branches are predictable. Selection has to re-scan the whole unsorted part on every pass.

- Measured (N = 10,000 ints):
 - random (one run): selection 49,995,000 comparisons, insertion 24,971,188 (about half)
 - sorted: selection 49,995,000, insertion 9,999

Worked steps / details

Alternative (textbook) version: find the MIN and swap it to the FRONT. Either version is accepted; the lecture used find-max.

Pass	Min (index)	Swap with	Array after the pass	Comparisons	Swaps
1	2 (index 1)	a[0] = 10	2 10 12 5 3	4	1
2	3 (index 4)	a[1] = 10	2 3 12 5 10	3	1
3	5 (index 3)	a[2] = 12	2 3 5 12 10	2	1
4	10 (index 4)	a[3] = 12	2 3 5 10 12	1	1
Total			2 3 5 10 12	10	4

Insertion sort on the same array (for the comparison; the sorted part is in bold)

Pass	Insert	Array after the pass	Comparisons	Shifts
1	2	2 10 12 5 3	1	1
2	12	2 10 12 5 3	1	0
3	5	2 5 10 12 3	3	2
4	3	2 3 5 10 12	4	3
Total			9	6

Element assignments:

- selection: 4 swaps x 3 = 12
- insertion: 6 shifts + 4 key saves + 4 key writes = 14

Counts for 5 elements in different orders (checked with a program; a swap is counted only when max_loc != N-1-i, as in the lecture)

Data	Selection: comparisons / swaps	Insertion: comparisons / shifts
10 2 12 5 3 (given)	10 / 4	9 / 6
2 3 5 10 12 (sorted)	10 / 0	4 / 0
12 10 5 3 2 (reverse)	10 / 2	10 / 10

Larger N (g++ -O2; times depend on the machine; random = one run)

Data (N = 10,000 ints)	Selection comparisons	Insertion comparisons	Selection time	Insertion time
sorted	49,995,000	9,999	142 ms	0.03 ms
random	49,995,000	24,971,188	144 ms	19 ms
reverse	49,995,000	49,995,000 (+ 49,995,000 shifts)	146 ms	38 ms

With 512-byte records (N = 4,000), where every move is expensive:

- reverse: selection about 26 ms, insertion about 160 to 220 ms
- random: selection about 26 ms, insertion about 80 to 90 ms
- sorted: selection about 26 ms, insertion about 1 ms

So which one is faster depends on the order of the data **and** on how expensive it is to move an element. With small int keys, insertion's shifts are cheap and sequential, so it was still faster even on reverse data.

الشرح بالعربي خطوة بخطوة

المطلوب: ترتيب المصفوفة التالية تصاعدياً بالاختيار:

10 2 12 5 3

طريقة الدكتور: في كل مرور نبحث عن أكبر عنصر في الجزء غير المرتب، ونبدله مع آخر خانة في هذا الجزء. هكذا يكبر الجزء المرتب من جهة اليمين.

المرور الأول: نبحث في المصفوفة كلها. الأكبر هو 12 في الخانة رقم 2، فنبدله مع آخر خانة (قيمتها 3):

10 2 3 5 12

أربع مقارنات وتبديل واحد.

المرور الثاني: نبحث في أول أربع خانات. الأكبر 10 في الخانة 0، فنبدله مع الخانة 3 (قيمتها 5):

5 2 3 10 12

ثلاث مقارنات وتبديل واحد.

المرور الثالث: نبحث في أول ثلاث خانات. الأكبر 5 في الخانة 0، فنبدله مع الخانة 2 (قيمتها 3):

3 2 5 10 12

مقارنتان وتبديل واحد.

المرور الرابع: نبحث في أول خانتين. الأكبر 3، فنبدله مع الخانة 1:

2 3 5 10 12

مقارنة واحدة وتبديل واحد.

المرور الخامس: بقي عنصر واحد، فهو مرتب.

المجموع: 10 مقارنات و4 تبديلات.

ملاحظة: إذا كان الأكبر في مكانه الصحيح أصلاً فلا نحسب تبديلاً، كما في المرور الرابع في مثال المحاضرة. والطريقة المشهورة في الكتب تبحث عن الأصغر وتضعه في البداية، وتعطي أيضاً 10 مقارنات و4 تبديلات. الطريقتان مقبولتان، لكن الدكتور استخدم طريقة الأكبر.

لماذا يختلف الزمن الفعلي بين الاختيار والإدراج مع أن كليهما من رتبة n^2 ؟

- الرتبة تخبرنا فقط كيف يكبر الزمن مع كبر n ، وتهمل الثوابت وتأثير ترتيب البيانات.

- الاختيار لا يهتم بالبيانات: يعمل دائماً $n(n-1)/2$ مقارنة (10 مقارنات لخمس عناصر)، سواء كانت البيانات مرتبة أو معكوسة. لكن تبديلاته قليلة جداً ($n-1$ على الأكثر).

- الإدراج يعتمد على البيانات: عمله يقارب عدد الأزواج المقلوبة. إذا كانت البيانات مرتبة يحتاج 4 مقارنات فقط بدون أي إزاحة، وإذا كانت معكوسة يحتاج 10 مقارنات و10 إزاحات.

- في مصفوفتنا 6 أزواج مقلوبة، فاحتاج الإدراج 9 مقارنات و6 إزاحات، واحتاج الاختيار 10 مقارنات و4 تبديلات. أي أنهما متقاربان جداً هنا.

- التبديل الواحد يساوي 3 عمليات نسخ، والإزاحة الواحدة عملية نسخ واحدة. إذا كانت العناصر كبيرة الحجم (سجلات) يصبح النقل مكلفاً، فيفوز الاختيار مع البيانات العشوائية والمعكوسة لأن تبديلاته قليلة. أما مع البيانات المرتبة أو شبه المرتبة فيفوز الإدراج بفارق كبير.

- الذاكرة المؤقتة السريعة

cache

تحب الوصول المتتالي. الإدراج يعمل على عناصر متجاورة لمسها قبل قليل ويتوقف مبكراً، بينما الاختيار يعيد المرور على الجزء غير المرتب كله في كل مرة.

- في تجربة بالبرنامج على 10000 عنصر عشوائي: عمل الاختيار 49995000 مقارنة، وعمل الإدراج تقريباً نصفها (24971188 مقارنة).

Lecture link: 8/27 selection sort page: the instructor's find-max version. It has a P1..P5 trace of 5 4 8 1 3 with comparison and swap counts per pass (4/1, 3/1, 2/1, 1/0), max_loc = 0, inner loop from 1 to N-1-i, and swap of a[max_loc] with a[N-1-i]; comparisons $\sim N^2$ so $O(N^2)$, swaps $\sim N$. The lecture code swaps unconditionally but counts P4 (max already in place) as 0 swaps; the 'if (max_loc != N-1-i)' in the code

here makes that convention explicit. 8/27 insertion sort page (sorted | unsorted, insert by shifting) is used for the runtime comparison.

L3. Write the code for the following class in C++. What additional code may be needed to ensure operation. Customer: - name : string; - phone : string; - home : Address; + Customer(name : string, phone : string, home : Address); + print(ostream&) : ostream&

L4. Provide a design for the Address class that holds a US style address, including any methods that may be needed by other classes, such as Customer, using the Address class.

Answer (written part)

L3: What additional code is needed for `Customer` to work?

1. **The Address class must be declared and defined (address.h / address.cpp), and customer.h must #include "address.h".** Customer holds an Address *by value* (Address home;), so the compiler needs the complete class to know sizeof(Customer) . A forward declaration (class Address;) is **not** enough; it would only work for an Address* or Address& member. Tried: error: field 'home' has incomplete type 'Address' .
2. **Address must provide std::ostream& print(std::ostream&) const (or operator<<), and it must be const.** Customer::print has to output the address, but the address fields are private to Address , so Customer::print calls home.print(out) and returns out . Because Customer::print is const , home is a const Address inside it and only const methods can be called on it. Tried with a non-const Address::print: error: passing 'const Address' as 'this' argument discards qualifiers .
3. **Address needs a usable copy constructor and operator= .** The constructor takes Address home by value, which is one copy, and home(home) in the initializer list copies it again into the member. Customer c2(c1) and c3 = c1 also copy home . The compiler-generated copy is correct here, because every member is a std::string: there are no raw pointers, so no deep copy is needed. The design writes both out anyway, to show what Customer relies on, and because Address declares a destructor. Since C++11 the implicit copy of such a class is deprecated. This is the rule of three: if you write one of the destructor, copy constructor or operator= , write all three.
4. **A default constructor Address() is needed** if the Customer constructor sets home inside its body (this->home = home;) instead of in the initializer list. home is then default-constructed first, and without Address() the compiler reports error: no matching function for call to 'Address::Address()' . In the same way, Customer has no default constructor, because it declares one with parameters. Customer c; or Customer list[10]; fails with no matching function for call to 'Customer::Customer()' . Allowing that needs a Customer() , which in turn needs Address() .
5. **Include guards (#ifndef ADDRESS_H_ / #define ADDRESS_H_ / #endif, and the same for CUSTOMER_H_).** main.cpp includes address.h directly and again through customer.h . Without the guards the compiler reports error: redefinition of 'class Address' .
6. **#include <string> and #include <iostream>**, with std::string and std::ostream written out in the headers. There is no using namespace std; in a header.

7. **A driver** `main()` to test the classes, and a Makefile whose `customer.o` and `main.o` rules list `address.h` as a dependency. Getters and setters for `Customer` are optional; the UML has none, so a `Customer` can only be printed.

Efficiency note: passing the strings and the `Address` by `const &` would avoid a copy. The UML specifies pass by value, so the code follows it.

L4: Address design

```
+-----+
|                                     |
|                                     |
+-----+
| - street : string  // line 1: "123 Main Street" |
| - unit   : string  // line 2: "Apt 4B", "" if none (optional) |
| - city   : string  // "Miami" |
| - state  : string  // 2 letter USPS code: "FL" |
| - zip    : string  // "33146" or ZIP+4 "33146-0620" |
+-----+
| + Address() |
| + Address(street, city, state, zip : string) |
| + Address(street, unit, city, state, zip : string) |
| + Address(other : const Address&) // copy constructor |
| + operator=(other : const Address&) : Address& |
| + ~Address() // virtual |
| + getStreet() : string ... getZip() : string {const} |
| + setStreet(s : string) ... setZip(z : string) |
| + isValid() : bool {const} |
| + print(out : ostream&) : ostream& {const} |
| - validState() : bool {const} |
| - validZip() : bool {const} |
+-----+
operator<<(out : ostream&, a : const Address&) : ostream& (free function)
```

Justification for each part

- **All fields are string.** `zip` is a string because zip codes can start with 0 (02134) and ZIP+4 contains a dash. The fields are private for information hiding.
- **Address()**: needed whenever an `Address` is default-constructed, for example as an array element, as a member assigned in a constructor body, or inside a future `Customer()`. Its placeholder values ("none", state "--") are deliberately invalid.
- **Two value constructors:** line 2 (the unit) is optional in US addresses, so there is one constructor with it and one without.
- **Copy constructor and operator=:** `Customer` needs them for the by-value parameter, for `home(home)`, and for copying or assigning `Customers`.
- **Virtual destructor:** follows the course style. It is empty because the strings free themselves.
- **Getters and setters:** let other classes read or update parts of an address (for example, sort customers by zip, or change a unit) without exposing the data.
- **isValid()**: checks that street and city are non-empty, that the state is exactly 2 uppercase letters, and that the zip is NNNNN or NNNNN-NNNN. It checks the format only; checking that a state actually exists would need a table of the USPS codes. The helpers are private because only the class uses them.

- **print(std::ostream&) const returning the stream:** the method `Customer::print` needs. It must be `const` because `Customer::print` is `const`. It prints in mailing-label layout and skips the unit line when it is empty. Returning the stream allows chaining and works with `cout`, `cerr` or a file.
- **operator<<:** convenience so `std::cout << addr;` works. It just returns `a.print(out)`.

Verification

- g++ 13.3 and clang++ 18.1 with `-std=c++11 -Wall -Wextra -pedantic:0` warnings.
- `isValid()` was checked against 31 cases: valid 5-digit, leading-zero and ZIP+4 zips; lowercase, mixed-case and wrong-length states; letters, spaces and a misplaced dash in the zip; empty street or city; and the default Address. All gave the expected result.
- The demo shows that a Customer keeps its own copy of the Address: changing the original afterwards does not change `c1`. `Customer c2(c1)` and `c3 = c1` produce identical output.
- `valgrind`: All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors.
- With extra flags that are not part of the course set (`-Wdeprecated`), the compilers note that `Customer` has a destructor but uses the compiler-generated copy. That copy is correct. Adding a copy constructor and `operator=` to `Customer`, written like the ones in `Address`, would silence the note.

L3 code — Customer

customer.h

```
#ifndef CUSTOMER_H_
#define CUSTOMER_H_

#include <iostream>
#include <string>

// Customer holds an Address object (not a pointer), so the compiler
// needs the full Address class here - a forward declaration
// (class Address;) is NOT enough
#include "address.h"

class Customer {
private:
    std::string name;        // customer name
    std::string phone;       // customer phone number
    Address home;            // home address (Customer has-a Address)

public:
    // constructor
    Customer(std::string name, std::string phone, Address home);

    // destructor
    virtual ~Customer();

    // print the customer: name, phone and address
    std::ostream& print(std::ostream&) const;
};

#endif
```

customer.cpp

```

#include "customer.h"

// definition of constructor
// using an initializer list to set the data
// home(home) copies the Address using the Address copy constructor
// param: name : string - customer name
// param: phone : string - customer phone number
// param: home : Address - home address (passed by value, so it is a copy)
Customer::Customer(std::string name, std::string phone, Address home)
    : name(name), phone(phone), home(home) {}

// destructor
// do not need code, name, phone and home clean up after themselves
Customer::~~Customer() {}

// print the data to output stream
// the Address prints itself, Customer does not need to know
// how an address is laid out
// this method is const, so Address::print must be const too
// param: out : ostream& - reference to output stream
// return: ostream& - output stream
std::ostream& Customer::print(std::ostream &out) const {
    out << "Customer name: " << name << std::endl;
    out << "Customer phone: " << phone << std::endl;
    out << "Customer address:" << std::endl;
    home.print(out);
    return out;
}

```

L4 code — Address

address.h

```

#ifndef ADDRESS_H_
#define ADDRESS_H_

#include <iostream>
#include <string>

// US style mailing address
// example:
//     123 Main Street      <- street (line 1)
//     Apt 4B              <- unit   (line 2, optional)
//     Miami, FL 33146     <- city, state zip
class Address {
private:
    std::string street;    // line 1: house number and street name
    std::string unit;      // line 2: apt / suite / room, "" if none
    std::string city;      // city name
    std::string state;     // 2 letter USPS state code, e.g. FL
    std::string zip;       // 5 digit zip (33146) or zip+4 (33146-0620)

    // check the state format
    bool validState() const;

    // check the zip format
    bool validZip() const;

public:
    // default constructor
    // needed so an Address can be created with no data
    // (e.g. an array of Address, or a class member not in an initializer list)
    Address();

```

```

    // constructor - address with no unit (line 2)
    Address(std::string street, std::string city,
            std::string state, std::string zip);

    // constructor - address with a unit (line 2)
    Address(std::string street, std::string unit, std::string city,
            std::string state, std::string zip);

    // copy constructor
    // used when an Address is passed by value, and by
    // Customer's initializer list home(home)
    Address(const Address&);

    // assignment operator
    // used when a Customer is assigned: c3 = c1 copies c1.home
    Address& operator=(const Address&);

    // destructor
    virtual ~Address();

    // get the data
    std::string getStreet() const;
    std::string getUnit() const;
    std::string getCity() const;
    std::string getState() const;
    std::string getZip() const;

    // set the data
    void setStreet(std::string);
    void setUnit(std::string);
    void setCity(std::string);
    void setState(std::string);
    void setZip(std::string);

    // check that the address is complete and well formed
    bool isValid() const;

    // print the address
    // const: Customer::print is const, so it can only call
    // const methods on its home member
    std::ostream& print(std::ostream&) const;
};

// output operator, so we can write: std::cout << address;
std::ostream& operator<<(std::ostream&, const Address&);

#endif

```

address.cpp

```

#include "address.h"

// definition of default constructor
// using an initializer list to set the defaults
// "--" is not a valid state, so a default Address is not valid
// until the real data is set
Address::Address() : street("none"), unit(""), city("none"),
                    state("--"), zip("00000") {}

// definition of constructor - address with no unit
// param: street : string - house number and street name
// param: city : string - city name

```

```

// param: state : string - 2 letter state code
// param: zip : string - zip or zip+4
Address::Address(std::string street, std::string city,
                 std::string state, std::string zip)
    : street(street), unit(""), city(city), state(state), zip(zip) {}

// definition of constructor - address with a unit
// param: street : string - house number and street name
// param: unit : string - apt / suite / room
// param: city : string - city name
// param: state : string - 2 letter state code
// param: zip : string - zip or zip+4
Address::Address(std::string street, std::string unit, std::string city,
                 std::string state, std::string zip)
    : street(street), unit(unit), city(city), state(state), zip(zip) {}

// copy constructor
// makes a new Address with the same data as other
// all members are std::string, so copying each one is a full copy
// (no pointers, so no deep copy code is needed)
// param: other : const Address& - address to copy
Address::Address(const Address &other)
    : street(other.street), unit(other.unit), city(other.city),
      state(other.state), zip(other.zip) {}

// assignment operator
// copies the data of other into this Address
// param: other : const Address& - address to copy
// return: Address& - this Address, so a = b = c works
Address& Address::operator=(const Address &other) {
    if (this != &other) { // a = a: nothing to do
        street = other.street;
        unit = other.unit;
        city = other.city;
        state = other.state;
        zip = other.zip;
    }
    return *this;
}

// destructor
// do not need code, all members are std::string
// and clean up after themselves
Address::~Address() {}

// get the street
// param: none
// return: string - street (line 1)
std::string Address::getStreet() const {
    return street;
}

// get the unit
// param: none
// return: string - unit (line 2), "" if none
std::string Address::getUnit() const {
    return unit;
}

// get the city
// param: none
// return: string - city
std::string Address::getCity() const {
    return city;
}

```

```

// get the state
// param: none
// return: string - 2 letter state code
std::string Address::getState() const {
    return state;
}

// get the zip
// param: none
// return: string - zip or zip+4
std::string Address::getZip() const {
    return zip;
}

// set the street
// param: street : string - house number and street name
// return: none
void Address::setStreet(std::string street) {
    this->street = street;
}

// set the unit
// param: unit : string - apt / suite / room, "" for none
// return: none
void Address::setUnit(std::string unit) {
    this->unit = unit;
}

// set the city
// param: city : string - city name
// return: none
void Address::setCity(std::string city) {
    this->city = city;
}

// set the state
// param: state : string - 2 letter state code
// return: none
void Address::setState(std::string state) {
    this->state = state;
}

// set the zip
// param: zip : string - zip or zip+4
// return: none
void Address::setZip(std::string zip) {
    this->zip = zip;
}

// check the state: exactly 2 uppercase letters, e.g. FL
// only checks the format, not that the state really exists
// param: none
// return: bool - true if the format is ok
bool Address::validState() const {
    if (state.length() != 2)
        return false;
    for (std::string::size_type i=0; i<state.length(); i++) {
        if (state[i] < 'A' || state[i] > 'Z')
            return false;
    }
    return true;
}

// check the zip: NNNNN or NNNNN-NNNN

```

```

// param: none
// return: bool - true if the format is ok
bool Address::validZip() const {
    if (zip.length() != 5 && zip.length() != 10)
        return false;
    for (std::string::size_type i=0; i<zip.length(); i++) {
        if (i == 5) {
            // zip+4: the 6th character must be the dash
            if (zip[i] != '-')
                return false;
        } else if (zip[i] < '0' || zip[i] > '9') {
            return false;
        }
    }
    return true;
}

// check that the address is complete and well formed
// unit is optional so it is not checked
// param: none
// return: bool - true if the address is valid
bool Address::isValid() const {
    return !street.empty() && !city.empty() && validState() && validZip();
}

// print the address to output stream, US mailing label format
// the unit line is only printed if there is one
// param: out : ostream& - reference to output stream
// return: ostream& - output stream
std::ostream& Address::print(std::ostream &out) const {
    out << street << std::endl;
    if (!unit.empty())
        out << unit << std::endl;
    out << city << ", " << state << " " << zip << std::endl;
    return out;
}

// output operator, so we can write: std::cout << address;
// param: out : ostream& - reference to output stream
// param: a : const Address& - address to print
// return: ostream& - output stream
std::ostream& operator<<(std::ostream &out, const Address &a) {
    return a.print(out);
}

```

Test driver

main.cpp

```

#include <iostream>
#include <string>

// both headers include address.h - the include guards
// stop the Address class from being defined twice
#include "address.h"
#include "customer.h"

int main(int argc, char *argv[]) {

    // 2 arguments
    // 1. customer name
    // 2. customer phone
    if (argc!=3) {
        std::cerr << "Usage: " << argv[0] << " <name> <phone>\n";
    }
}

```

```

    return -1;
}

// create an Address with the default constructor
Address blank;
std::cout << "++++++\n";
std::cout << "default Address:\n";
blank.print(std::cout);
std::cout << "valid: " << (blank.isValid() ? "yes" : "no") << std::endl;

// create an Address with the full constructor (with a unit)
Address a("123 Main Street", "Apt 4B", "Miami", "FL", "33146");
std::cout << "++++++\n";
std::cout << "Address a:\n";
std::cout << a;    // same as a.print(std::cout)
std::cout << "valid: " << (a.isValid() ? "yes" : "no") << std::endl;

// create a Customer from the command line and Address a
// a is passed by value, so c1 gets its own copy in home
Customer c1(argv[1], argv[2], a);
std::cout << "++++++\n";
std::cout << "Customer c1:\n";
c1.print(std::cout);

// change Address a - c1 does not change
// because c1.home is a copy, not the same object
a.setStreet("500 Ocean Drive");
a.setUnit("");
a.setZip("33139-1234");
std::cout << "++++++\n";
std::cout << "Address a after the set calls:\n";
a.print(std::cout);
std::cout << "valid: " << (a.isValid() ? "yes" : "no") << std::endl;
std::cout << "Customer c1 is unchanged:\n";
c1.print(std::cout);

// copy a Customer with the (compiler supplied) copy constructor
Customer c2(c1);
std::cout << "++++++\n";
std::cout << "Customer c2 (copy of c1):\n";
c2.print(std::cout);

// copy a Customer with the (compiler supplied) assignment operator
Customer c3("John Doe", "305-555-0199",
            Address("1 Palm Avenue", "Orlando", "FL", "32801"));
std::cout << "++++++\n";
std::cout << "Customer c3 before c3 = c1:\n";
c3.print(std::cout);
c3 = c1;
std::cout << "Customer c3 after c3 = c1:\n";
c3.print(std::cout);

// an Address with a bad state and a bad zip
Address bad("9 Elm Street", "Miami", "Florida", "3314");
std::cout << "++++++\n";
std::cout << "Address bad:\n";
bad.print(std::cout);
std::cout << "valid: " << (bad.isValid() ? "yes" : "no") << std::endl;
std::cout << "++++++\n";

return 0;
}

```

Build and run:

```
cd /home/user/SYATRAT/ECE218/Sample1 && make          # g++ -std=c++11 -Wall -Wextra -pedantic -g
(clang: make clean && make CXX=clang++)
cd /home/user/SYATRAT/ECE218/Sample1 && ./customer "Jane Smith" "305-555-0123"      # or: make run ;
make memcheck (valgrind --leak-check=full)
```

Real output:

```
$ ./customer
Usage: ./customer <name> <phone>
exit: 255
$ ./customer "Jane Smith" "305-555-0123"
+++++
default Address:
none
none, -- 00000
valid: no
+++++
Address a:
123 Main Street
Apt 4B
Miami, FL 33146
valid: yes
+++++
Customer c1:
Customer name: Jane Smith
Customer phone: 305-555-0123
Customer address:
123 Main Street
Apt 4B
Miami, FL 33146
+++++
Address a after the set calls:
500 Ocean Drive
Miami, FL 33139-1234
valid: yes
Customer c1 is unchanged:
Customer name: Jane Smith
Customer phone: 305-555-0123
Customer address:
123 Main Street
Apt 4B
Miami, FL 33146
+++++
Customer c2 (copy of c1):
Customer name: Jane Smith
Customer phone: 305-555-0123
Customer address:
123 Main Street
Apt 4B
Miami, FL 33146
+++++
Customer c3 before c3 = c1:
Customer name: John Doe
Customer phone: 305-555-0199
Customer address:
1 Palm Avenue
Orlando, FL 32801
Customer c3 after c3 = c1:
Customer name: Jane Smith
Customer phone: 305-555-0123
Customer address:
123 Main Street
Apt 4B
```

```

Miami, FL 33146
+++++
Address bad:
9 Elm Street
Miami, Florida 3314
valid: no
+++++
exit: 0
$ valgrind --leak-check=full ./customer "Jane Smith" "305-555-0123"
==4458== All heap blocks were freed -- no leaks are possible
==4458== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
(g++ 13.3.0 and clang++ 18.1.3 with -std=c++11 -Wall -Wextra -pedantic: 0 warnings, 0 errors.
valgrind 3.22 is clean for both builds, for both the normal run and the Usage path.)

```

الشرح بالعربي خطوة بخطوة

الفكرة العامة

السؤالان مرتبطان: في السؤال الثالث نكتب صنف العميل، وفي السؤال الرابع نصمم صنف العنوان الذي يستخدمه العميل. العلاقة بينهما علاقة «يملك»: كل عميل يملك عنوانًا، وهذا يسمّى التركيب. العنوان مخزن داخل كائن العميل نفسه كقيمة كاملة، وليس مؤشرًا إلى مكان آخر في الذاكرة. هذه النقطة هي مفتاح الجواب عن سؤال «ما الشيفرة الإضافية المطلوبة».

الخطوة ١: نبدأ بالعنوان لأن العميل يعتمد عليه

العنوان الأمريكي فيه خمسة أجزاء:

- الشارع: رقم البيت واسم الشارع.
- الوحدة: رقم الشقة أو المكتب، وهي اختيارية.
- المدينة.
- الولاية: رمز من حرفين كبيرين، مثل

FL

- الرمز البريدي: خمسة أرقام، أو خمسة أرقام ثم شرطة ثم أربعة، مثل

33146-0620

نخزن كل الأجزاء نصوصًا ونجعلها خاصة، وهذا هو إخفاء المعلومات.

سؤال مهم: لماذا الرمز البريدي نص وليس عددًا صحيحًا؟ لأن بعض الرموز تبدأ بصفر، ولو خزّناه عددًا لضاع هذا الصفر. والشكل الطويل فيه شرطة، والشرطة ليست رقمًا أصلًا.

الخطوة ٢: الدوال التي يحتاجها العنوان (جواب السؤال الرابع)

- باني افتراضي بدون معاملات. قيمه غير صالحة عن قصد، فالولاية فيه «---» إلى أن نضع البيانات الحقيقية.
- بانين كاملان، واحد بدون الوحدة وواحد معها. هذا اسمه التحميل الزائد للبانين.

- باني النسخ وعامل الإسناد. العميل يحتاجهما، لأن العنوان يُمرَّر إلى باني العميل بالقيمة فيُنسخ، وعند نسخ عميل أو إسناده إلى عميل آخر يُنسخ عنوانه أيضًا.

- هادم ظاهري على طريقة الأستاذ، أي مكتوب قبله الكلمة

virtual

وجسمه فارغ، لأن النصوص تحرَّر ذاكرتها بنفسها.

- دوال قراءة وتعديل لكل جزء، حتى تستخدم الأصناف الأخرى أجزاء العنوان بدون كسر الإخفاء، مثل ترتيب العملاء حسب الرمز البريدي.

- دالة تحقق تفحص أن الشارع والمدينة غير فارغين، وأن الولاية حرفان كبيران بالضبط، وأن الرمز البريدي خمسة أرقام، أو خمسة ثم شرطة ثم أربعة. هي تفحص الشكل فقط، ولا تتأكد أن الولاية موجودة فعليًا. جربناها على أكثر من عشرين حالة صحيحة وخاطئة، وكانت النتيجة صحيحة في كل الحالات.

- أهم دالة بالنسبة للعميل هي دالة الطباعة. تأخذ مرجعًا إلى مجرى الإخراج، وترجع نفس المرجع:

```
std::ostream& print(std::ostream&) const
```

إرجاع المجري يسمح بالطباعة المتسلسلة، ونفس الدالة تعمل مع الشاشة أو مع ملف. والكلمة الأخيرة

const

معناها أن الدالة ثابتة لا تغيّر الكائن. سنرى في الخطوة ٤ لماذا هذا ضروري.

- أضفنا عامل الإخراج للراحة فقط، وهو يستدعي دالة الطباعة:

```
operator<<
```

الخطوة ٣: صنف العميل كما في المخطط تمامًا (السؤال الثالث)

- ثلاثة أعضاء خاصة: الاسم والهاتف نصوص، والعنوان كائن من صنف العنوان.

- الباني يستخدم قائمة التهيئة، مثل:

```
home(home)
```

الاسم خارج القوسين هو العضو، والاسم داخلهما هو المعامل.

- لماذا قائمة التهيئة مهمة هنا؟ لأن العنوان يُبنى مباشرة نسخةً من المعامل. لو كتبنا الإسناد داخل جسم الباني، فالمترجم يبني العنوان أولاً بالباني الافتراضي ثم ينسخ فوقه، وإذا لم يوجد باني افتراضي تفشل الترجمة. جربنا هذا فعليًا وظهر الخطأ.

- دالة الطباعة تطبع الاسم والهاتف، ثم تطلب من العنوان أن يطبع نفسه، ثم ترجع المجري:

العميل لا يحتاج أن يعرف كيف يُرتَّب العنوان على الورق، وهذا تقسيم جميل للمسؤوليات.

- أضفنا هادماً ظاهرياً على طريقة الأستاذ، وهذه هي الإضافة الوحيدة على المخطط.

الخطوة ٤: ما الشيفرة الإضافية المطلوبة حتى يعمل الصنف؟

١. صنف العنوان نفسه، مع تضمين ملف رأسه داخل ملف رأس العميل. التصريح المسبق وحده لا يكفي، لأن المترجم يحتاج أن يعرف حجم العنوان حتى يحسب حجم العميل. هذا الخطأ الذي ظهر لنا:

```
incomplete type 'Address'
```

التصريح المسبق يكفي فقط لو كان العضو مؤشراً أو مرجعاً.

٢. دالة طباعة في العنوان، لأن حقوله خاصة والعميل لا يقدر يوصل لها. ويجب أن تكون دالة ثابتة: دالة طباعة العميل ثابتة، وداخلها يصير العنوان ثابتاً، فلا يمكن أن نستدعي عليه إلا دوال ثابتة. جرّبنا حذف الثبات فظهر خطأ في الترجمة:

```
discards qualifiers
```

٣. باني نسخ وعامل إسناد صالحان في العنوان. النسخة التي يولدها المترجم صحيحة هنا، لأن كل الحقول نصوص ولا يوجد أي مؤشر، فلا نحتاج نسخاً عميقاً. لكننا كتبناهما صراحةً لسببين. الأول أن نوضح ما يعتمد عليه العميل. والثاني قاعدة الثلاثة: إذا كتبت بنفسك واحداً من الثلاثة، أي الهادم أو باني النسخ أو عامل الإسناد، فاكتب الثلاثة كلها. ونحن كتبنا الهادم.

٤. باني افتراضي للعنوان، إذا أسندنا العنوان داخل جسم الباني بدل قائمة التهيئة. وانتبه: العميل نفسه ليس له باني افتراضي، لأننا كتبنا له بانياً بمعاملات. فلو أردنا عميلاً فارغاً أو مصفوفة من العملاء، يجب أن نضيف باني افتراضي للعميل، وهذا بدوره يحتاج الباني الافتراضي للعنوان.

٥. حرّاس التضمين، لأن الملف الرئيسي يضمن ملف العنوان مرتين: مرة مباشرة، ومرة من خلال ملف العميل. بدونها يظهر خطأ إعادة تعريف الصنف.

٦. تضمين مكتبة النصوص ومكتبة الإدخال والإخراج، واستخدام هذه البادئة في ملفات الرأس بدل فتح فضاء الأسماء كله:

```
std::
```

٧. برنامج رئيسي للاختبار، وملف بناء يذكر أن ملف العميل والملف الرئيسي يعتمدان على ملف رأس العنوان.

ملاحظة للامتحان: التمرير بمرجع ثابت أسرع لأنه يوفر نسخة، لكن المخطط يقول بالقيمة فالتزمنا به.

الخطوة ٥: البرنامج الرئيسي

- يتحقق من عدد المعاملات. إذا كان العدد خطأ، يطبع سطر الاستخدام في مجرى الأخطاء ويرجع سالب واحد، مثل برنامج الترتيب عند الأستاذ.

- يبني عنوانًا، ثم يبني منه عميلًا ويطبعه.
- يغيّر العنوان الأصلي بعد بناء العميل، والعميل لا يتغيّر. هذا دليل أن العميل عنده نسخته الخاصة.
- ينسخ العميل بباني النسخ وبالإسناد، والنسختان مطابقتان للأصل.
- يطبع عنوانًا خاطئًا، والتحقق يرجع «لا».

الخطوة ٦: الاختبار

- بنينا البرنامج بالترجمين الاثنيين مع خيارات التحذير الصارمة، ولم يظهر أي تحذير.
- أداة فحص الذاكرة

valgrind

أظهرت صفر أخطاء، ولا يوجد أي تسريب للذاكرة. هذا متوقع، لأننا لم نحجز شيئًا في الكومة بأنفسنا: الكائنات على المكس، والنصوص تدير ذاكرتها بنفسها.

الخلاصة للامتحان

اكتب صنف العميل مثل المخطط، مع قائمة تهيئة ودالة طباعة ترجع المجرى. ثم عدّد الشيفرة الإضافية: صنف العنوان وتضمن ملفه، ودالة طباعة ثابتة للعنوان، وباني النسخ وعامل الإسناد، والباني الافتراضي، وحرّاس التضمنين، وبرنامج رئيسي للاختبار. وفي السؤال الرابع ارسم صندوق الصنف: الحقول الخاصة بعلامة الناقص، والدوال العامة بعلامة الزائد، واكتب سطرًا قصيرًا يبرّر كل دالة.

Part 3 — Practice 1: Simple Smart Home

Implement the initial Smart Home: 1 Room with Lights. Room (id, name, location, lights[5], numLights; setLightOn(index) : int, setLightOff(index) : int, getters/setters). Light (id, name, state = false; setOn() : int, setOff() : int, isOn() : bool, getters/setters). Files room.h/cpp, light.h/cpp, main.cpp. Read the data file, then print the Room (1) initially, (2) with all lights ON, (3) with all lights OFF.

Design decisions

- Attributes and methods follow the UML exactly. All attributes are private and all methods are public. Light has int id, std::string name and bool state. Room has int id, std::string name, std::string location, Light lights[MAX_LIGHTS] (MAX_LIGHTS = 5) and int numLights. The required methods are setOn(): int, setOff(): int, isOn(): bool, setLightOn(int): int and setLightOff(int): int.
- Light's state defaults to false (OFF). Both Light constructors set state(false) in the initializer list, which matches 'state: bool = false' and the instructor's initializer-list style.
- Return-code convention: 0 = success, -1 = error, stated in both class headers and above every method. Light::setOn()/setOff() always return 0, because a single light cannot fail. Room::setLightOn/setLightOff/setLight return -1 when the index is outside 0..numLights-1. setNumLights rejects values outside 0..5 and leaves the count unchanged. addLight returns -1 when the room is full. So numLights can never exceed 5 and a valid index is always 0..4, which is now written in the comments of setLightOn/setLightOff and in the README.
- Room has getters and setters for every attribute: getId/setId, getName/setName, getLocation/setLocation, getNumLights/setNumLights, and getLight(int)/setLight(int, const Light&) for the array, all bounds-checked. There is also an addLight(const Light&) helper. getLight returns a copy, or a default Light (id -1) for a bad index. Light has getId/setId and getName/setName. Its state changes only through setOn/setOff and is read through isOn, as the UML specifies.
- const int MAX_LIGHTS = 5 is declared in room.h before the class. A namespace-scope const has internal linkage, so including room.h in several files is safe. Each array element is default-constructed automatically.
- No dynamic memory: the lights array is inside the Room object, which is a local variable in main, so the destructors are empty. valgrind reports 0 errors and all heap blocks freed.
- Instructor style is matched: #ifndef X_H_ include guards, std:: prefixes and no 'using namespace std' anywhere, indented private:/public: sections, unnamed parameters in the header declarations, virtual destructors, 'std::ostream& print(std::ostream&)' returning the stream, the '// param: x : type - desc' / '// return:' comment format, this->name = name in setters, '+++' separator lines, and the sort.cpp-style argc check that prints a Usage line to std::cerr and returns -1.
- The parser is a free function in main.cpp, int loadRoom(std::istream&, Room&). It reads tokens with >> in the exact file order (Room / / Light / n x ' ' / End), checks in.fail() and the expected keyword at each step, and fills the Room through its setters and addLight. Nothing is hard-coded.
- More than 5 lights (for example 'Light 7'): every listed light is still read so the parser stays in step with the file and reaches 'End'. The first 5 are kept and a warning goes to std::cerr for each ignored light. Every malformed input prints an error and makes main return -1.

- Blinds discrepancy: output step 3 says 'and all blinds to CLOSED', but the design has no blinds. No Blind class was invented. Step 3 turns every light OFF and prints the Room, its header says '(no blinds in this version)', and a comment in main.cpp and a note in README.md explain why.

Data file

room.txt

```
Room
1 LivingRoom FirstFloor
Light 4
101 Ceiling
102 FloorLamp
103 TableLamp
104 WallSconce
End
```

Code

light.h

```
#ifndef LIGHT_H_
#define LIGHT_H_

#include <iostream>
#include <string>

// Light - one light in a room of the Smart Home
//
// return code convention for methods that return int:
// 0 = success
// -1 = error
class Light {
private:
    int id;           // light id
    std::string name; // light name
    bool state;       // light state: true = ON, false = OFF (default false)

public:
    // constructor (default)
    Light();

    // constructor with id and name
    Light(int, std::string);

    // destructor
    virtual ~Light();

    // turn the light ON
    int setOn();

    // turn the light OFF
    int setOff();

    // check if the light is ON
    bool isOn();

    // getters and setters
    int getId();
    void setId(int);
}
```

```

        std::string getName();
        void setName(std::string);

        // print the light
        std::ostream& print(std::ostream&);

};

#endif

```

light.cpp

```

#include "light.h"

// definition of default constructor
// using an initializer list to set the defaults
// state = false means every new light starts OFF (UML: state: bool = false)
Light::Light() : id(-1), name("none"), state(false) {}

// definition of constructor with id and name
// using an initializer list to set the data
// the light still starts OFF
// param: id : int - light id to set
// param: name : string - light name to set
Light::Light(int id, std::string name) : id(id), name(name), state(false) {}

// destructor
// do not need code
Light::~Light() {}

// turn the light ON
// param: none
// return: int - 0 on success (turning a light ON cannot fail)
int Light::setOn() {
    state = true;
    return 0;
}

// turn the light OFF
// param: none
// return: int - 0 on success (turning a light OFF cannot fail)
int Light::setOff() {
    state = false;
    return 0;
}

// check if the light is ON
// param: none
// return: bool - true if the light is ON, false if it is OFF
bool Light::isOn() {
    return state;
}

// get the light id
// param: none
// return: int - light id
int Light::getId() {
    return id;
}

// set the light id
// param: id : int - light id to set
// return: none
void Light::setId(int id) {

```

```

    this->id = id;
}

// get the light name
// param: none
// return: string - light name
std::string Light::getName() {
    return name;
}

// set the light name
// param: name : string - light name to set
// return: none
void Light::setName(std::string name) {
    this->name = name;
}

// print the data to output stream
// param: out : ostream& - reference to output stream
// return: ostream& - output stream
std::ostream& Light::print(std::ostream &out) {
    out << "Light ID: " << id;
    out << "  name: " << name;
    if (state) {
        out << "  state: ON" << std::endl;
    } else {
        out << "  state: OFF" << std::endl;
    }
    return out;
}

```

room.h

```

#ifndef ROOM_H_
#define ROOM_H_

#include <iostream>
#include <string>

#include "light.h"

// maximum number of lights in a room (UML: lights[5])
const int MAX_LIGHTS = 5;

// Room - a room of the Smart Home (this version has only 1 Room with Lights)
//
// only lights[0] .. lights[numLights-1] are in use
//
// return code convention for methods that return int:
//   0 = success
//  -1 = error (for example an invalid light index)
class Room {
private:
    int id; // room id
    std::string name; // room name
    std::string location; // room location
    Light lights[MAX_LIGHTS]; // lights in the room (each one default constructed)
    int numLights; // number of lights in use

public:
    // constructor (default)
    Room();

    // constructor with id, name and location

```

```

Room(int, std::string, std::string);

// destructor
virtual ~Room();

// turn ON the light at index
int setLightOn(int);

// turn OFF the light at index
int setLightOff(int);

// getters and setters
int getId();
void setId(int);

std::string getName();
void setName(std::string);

std::string getLocation();
void setLocation(std::string);

int getNumLights();
int setNumLights(int);

Light getLight(int);
int setLight(int, const Light&);

// helper: add a light at the end of the array
int addLight(const Light&);

// print the room and all its lights
std::ostream& print(std::ostream&);

};

#endif

```

room.cpp

```

#include "room.h"

// definition of default constructor
// using an initializer list to set the defaults
// the array lights[MAX_LIGHTS] does not appear in the list:
// C++ calls Light() for every element, so each light starts id -1, "none", OFF
Room::Room() : id(-1), name("none"), location("none"), numLights(0) {}

// definition of constructor with id, name and location
// using an initializer list to set the data
// the room starts with no lights in use
// param: id : int - room id to set
// param: name : string - room name to set
// param: location : string - room location to set
Room::Room(int id, std::string name, std::string location)
    : id(id), name(name), location(location), numLights(0) {}

// destructor
// do not need code (lights is an array, not memory from new)
Room::~Room() {}

// turn ON the light at index
// numLights is never more than MAX_LIGHTS, so a valid index is
// always inside the array (0 .. 4)
// param: index : int - index of the light (0 .. numLights-1)

```

```

// return: int - 0 on success, -1 if index is invalid
int Room::setLightOn(int index) {
    if (index < 0 || index >= numLights) {
        return -1;
    }
    return lights[index].setOn();
}

// turn OFF the light at index
// numLights is never more than MAX_LIGHTS, so a valid index is
// always inside the array (0 .. 4)
// param: index : int - index of the light (0 .. numLights-1)
// return: int - 0 on success, -1 if index is invalid
int Room::setLightOff(int index) {
    if (index < 0 || index >= numLights) {
        return -1;
    }
    return lights[index].setOff();
}

// get the room id
// param: none
// return: int - room id
int Room::getId() {
    return id;
}

// set the room id
// param: id : int - room id to set
// return: none
void Room::setId(int id) {
    this->id = id;
}

// get the room name
// param: none
// return: string - room name
std::string Room::getName() {
    return name;
}

// set the room name
// param: name : string - room name to set
// return: none
void Room::setName(std::string name) {
    this->name = name;
}

// get the room location
// param: none
// return: string - room location
std::string Room::getLocation() {
    return location;
}

// set the room location
// param: location : string - room location to set
// return: none
void Room::setLocation(std::string location) {
    this->location = location;
}

// get the number of lights in use
// param: none
// return: int - number of lights in use

```

```

int Room::getNumLights() {
    return numLights;
}

// set the number of lights in use
// (making it larger shows the lights already stored in those
// slots, a default Light if the slot was never set)
// param: numLights : int - number of lights (0 .. MAX_LIGHTS)
// return: int - 0 on success, -1 if numLights is out of range (not changed)
int Room::setNumLights(int numLights) {
    if (numLights < 0 || numLights > MAX_LIGHTS) {
        return -1;
    }
    this->numLights = numLights;
    return 0;
}

// get a copy of the light at index
// param: index : int - index of the light (0 .. numLights-1)
// return: Light - copy of the light, or a default Light (id -1) if index is invalid
Light Room::getLight(int index) {
    if (index < 0 || index >= numLights) {
        return Light();
    }
    return lights[index];
}

// replace the light at index
// param: index : int - index of the light (0 .. numLights-1)
// param: light : const Light& - light to copy into the room
// return: int - 0 on success, -1 if index is invalid
int Room::setLight(int index, const Light &light) {
    if (index < 0 || index >= numLights) {
        return -1;
    }
    lights[index] = light;
    return 0;
}

// add a light at the end of the array
// param: light : const Light& - light to copy into the room
// return: int - 0 on success, -1 if the room is full (MAX_LIGHTS lights)
int Room::addLight(const Light &light) {
    if (numLights >= MAX_LIGHTS) {
        return -1;
    }
    lights[numLights] = light;
    numLights++;
    return 0;
}

// print the data to output stream
// param: out : ostream& - reference to output stream
// return: ostream& - output stream
std::ostream& Room::print(std::ostream &out) {
    out << "Room ID: " << id << std::endl;
    out << "Room name: " << name << std::endl;
    out << "Room location: " << location << std::endl;
    out << "Number of lights: " << numLights << std::endl;
    for (int i=0; i<numLights; i++) {
        out << "  lights[" << i << "] ";
        lights[i].print(out);
    }
}

```

```

    return out;
}

```

main.cpp

```

#include <iostream>
#include <fstream>
#include <string>

#include "light.h"
#include "room.h"

// load the room data from an input stream (the data file)
// expected format (tokens are separated by spaces / new lines):
//   Room
//   <id> <name> <location>
//   Light <number_of_lights>
//   <id> <name>          <- one line per light
//   ...
//   End
// names and locations must be single words (no spaces)
// a room holds at most MAX_LIGHTS lights: extra lights are read,
// a warning is printed and they are ignored
// param: in : istream& - reference to the input stream to read from
// param: room : Room& - reference to the room to fill with the data
// return: int - 0 on success, -1 on error (message printed to cerr)
int loadRoom(std::istream &in, Room &room) {
    std::string keyword;

    // 1. keyword "Room"
    in >> keyword;
    if (in.fail() || keyword != "Room") {
        std::cerr << "Error: expected keyword 'Room'\n";
        return -1;
    }

    // 2. room <id> <name> <location>
    int id;
    std::string name;
    std::string location;
    in >> id >> name >> location;
    if (in.fail()) {
        std::cerr << "Error: could not read room <id> <name> <location>\n";
        return -1;
    }
    room.setId(id);
    room.setName(name);
    room.setLocation(location);

    // 3. keyword "Light" and <number_of_lights>
    int count;
    in >> keyword;
    if (in.fail() || keyword != "Light") {
        std::cerr << "Error: expected keyword 'Light'\n";
        return -1;
    }
    in >> count;
    if (in.fail() || count < 0) {
        std::cerr << "Error: could not read a valid number of lights\n";
        return -1;
    }

    // 4. one light per line: <id> <name>
    for (int i=0; i<count; i++) {

```

```

    int lightId;
    std::string lightName;
    in >> lightId >> lightName;
    if (in.fail()) {
        std::cerr << "Error: could not read light " << i+1
                    << " of " << count << std::endl;
        return -1;
    }
    // create the Light object and copy it into the room
    if (room.addLight(Light(lightId, lightName)) != 0) {
        std::cerr << "Warning: a room holds at most " << MAX_LIGHTS
                    << " lights, ignoring light " << lightId << " "
                    << lightName << std::endl;
    }
}

// 5. keyword "End"
// if it is not found, the number of lights usually does not
// match the number of light lines in the file
in >> keyword;
if (in.fail() || keyword != "End") {
    std::cerr << "Error: expected keyword 'End' after " << count
                << " lights (check the number of lights)\n";
    return -1;
}

return 0;
}

int main(int argc, char *argv[]) {

    // 1 argument
    // 1. name of the data file
    if (argc!=2) {
        std::cerr << "Usage: " << argv[0] << " <data file>\n";
        return -1;
    }

    // open the data file
    std::ifstream inFile(argv[1]);
    if (!inFile.is_open()) {
        std::cerr << "Error: could not open data file " << argv[1] << std::endl;
        return -1;
    }

    // create a Room instance and load it with the data from the file
    Room room;
    if (loadRoom(inFile, room) != 0) {
        std::cerr << "Error reading data file " << argv[1] << std::endl;
        return -1;
    }
    inFile.close();

    // 1. print the initial state of the Room
    std::cout << "++++++\n";
    std::cout << "1. Initial state of the Room\n";
    std::cout << "++++++\n";
    room.print(std::cout);

    // 2. change the state of all lights to ON and print the Room
    for (int i=0; i<room.getNumLights(); i++) {
        if (room.setLightOn(i) != 0) {
            std::cerr << "Error: could not turn ON light " << i << std::endl;
        }
    }
}

```

```

std::cout << "++++++\n";
std::cout << "2. All lights ON\n";
std::cout << "++++++\n";
room.print(std::cout);

// 3. change the state of all lights to OFF and print the Room
// NOTE: the assignment text also says "and all blinds to CLOSED",
// but the UML design of this version has no blinds (only Room and
// Light), so only the lights are turned OFF here
for (int i=0;i<room.getNumLights();i++) {
    if (room.setLightOff(i) != 0) {
        std::cerr << "Error: could not turn OFF light " << i << std::endl;
    }
}
std::cout << "++++++\n";
std::cout << "3. All lights OFF (no blinds in this version)\n";
std::cout << "++++++\n";
room.print(std::cout);

return 0;
}

```

Makefile

```

# Makefile for the Simple Smart Home (ECE 218 Practice)
# make      -> builds the executable smarthome
# make clean -> removes the executable and object files

CXX = g++
CXXFLAGS = -std=c++11 -Wall -Wextra -pedantic

all: smarthome

smarthome: main.o room.o light.o
    $(CXX) $(CXXFLAGS) -o smarthome main.o room.o light.o

main.o: main.cpp room.h light.h
    $(CXX) $(CXXFLAGS) -c main.cpp

room.o: room.cpp room.h light.h
    $(CXX) $(CXXFLAGS) -c room.cpp

light.o: light.cpp light.h
    $(CXX) $(CXXFLAGS) -c light.cpp

clean:
    rm -f smarthome *.o

.PHONY: all clean

```

Build and run:

```

cd /home/user/SYATRAT/ECE218/Practice1 && g++ -std=c++11 -Wall -Wextra -pedantic -o smarthome
main.cpp room.cpp light.cpp (or: make)
cd /home/user/SYATRAT/ECE218/Practice1 && ./smarthome room.txt

```

Real output:

```

++++++
1. Initial state of the Room
++++++

```

```

Room ID: 1
Room name: LivingRoom
Room location: FirstFloor
Number of lights: 4
  lights[0] Light ID: 101  name: Ceiling  state: OFF
  lights[1] Light ID: 102  name: FloorLamp  state: OFF
  lights[2] Light ID: 103  name: TableLamp  state: OFF
  lights[3] Light ID: 104  name: WallSconce  state: OFF
+++++
2. All lights ON
+++++
Room ID: 1
Room name: LivingRoom
Room location: FirstFloor
Number of lights: 4
  lights[0] Light ID: 101  name: Ceiling  state: ON
  lights[1] Light ID: 102  name: FloorLamp  state: ON
  lights[2] Light ID: 103  name: TableLamp  state: ON
  lights[3] Light ID: 104  name: WallSconce  state: ON
+++++
3. All lights OFF (no blinds in this version)
+++++
Room ID: 1
Room name: LivingRoom
Room location: FirstFloor
Number of lights: 4
  lights[0] Light ID: 101  name: Ceiling  state: OFF
  lights[1] Light ID: 102  name: FloorLamp  state: OFF
  lights[2] Light ID: 103  name: TableLamp  state: OFF
  lights[3] Light ID: 104  name: WallSconce  state: OFF

```

Grading verdict

The draft already met every checklist item, so I found nothing to deduct points for. I made four small improvements, listed under `issues_fixed`: a clearer 'End' error message, a stated 0..4 index guarantee, a README fix with the real sample output, and three fixes to the Arabic explanation (an inaccurate include-guard claim, a bracket token that would render broken, and a vague phrase).

Verification (final state)

- **Build:** `g++ -std=c++11 -Wall -Wextra -pedantic` gives 0 warnings (g++ 13.3). `clang++ 18` with the same flags, `make` and `make CXX=clang++` also give 0 warnings, and so does g++ at -O2 or with `-std=c++98`. `make clean` was run afterwards, so the folder holds only the sources, `room.txt`, the Makefile and README.md.
- **Run on room.txt:** exit code 0. The output above is from the final binary, and the README's sample output was checked against it with `diff`.
- **Error paths** (all run, none crash):

Input	Result
no argument, or two arguments	Usage: ./smarthome <data file>, exit -1 (255)
nonexistent file	Error: could not open data file ..., exit -1
Light 7 with 7 light lines	keeps 5 lights, 2 warnings on stderr, runs normally
Light 7 with only 3 lines	Error: could not read light 4 of 7, exit -1
file cut off mid-light	Error: could not read light 3 of 4, exit -1
Light 2 with 3 lines, or missing End	Error: expected keyword 'End' after 2 lights (check the number of lights), exit -1
wrong keyword, non-numeric id, negative count, or empty file	error message, exit -1
Light 0	prints an empty room, exit 0

- **Class methods:** a scratch harness of 30 checks passed under valgrind (defaults, return codes, bounds at -1 and 5, full room, setNumLights 6 and -1 rejected). It is not part of the deliverable.
- **Memory:** valgrind --leak-check=full on room.txt, the harness and the malformed files gave ERROR SUMMARY: 0 errors, all heap blocks freed.

Note on the repository

I made no commits. While this review ran, another process in the same session committed my edits to Practice1 (README.md, main.cpp, room.cpp) as 13581ee "Update ECE218 solution drafts from verification pass", followed by 8fd8570 (a .gitignore fix). The working tree for Practice1 now matches HEAD exactly. The only uncommitted changes in the repo are in ECE218/Sample1 (address.h, customer.cpp), which this review did not touch.

الشرح بالعربي خطوة بخطوة

الخطوة الأولى: نقرأ المخطط قبل كتابة أي سطر

- في المخطط صنفان: صنف الغرفة وصنف الإنارة. الغرفة لها رقم واسم وموقع ومصفوفة من خمس إنارات، ومعها عدد الإنارات المستخدمة فعلاً.
- علامة الناقص قبل الصفة تعني أنها خاصة، فلا تصل إليها إلا دوال الصنف نفسه. هذا هو مبدأ إخفاء المعلومات، ولهذا نحتاج دوال جلب ودوال تعيين.
- الدوال ليس قبلها علامة الناقص، فنجعلها عامة حتى تستطيع الدالة الرئيسية استدعاءها.
- العبارة التالية تعني أن الحالة الافتراضية للإنارة هي «مطفأة»، فكل إنارة جديدة تبدأ مطفأة:

```
state: bool = false
```

- وهذه تعني مصفوفة ثابتة الحجم فيها خمسة كائنات من صنف الإنارة، وهي موجودة داخل كائن الغرفة نفسه وليست في مكان آخر:

```
Light lights[5];
```

الخطوة الثانية: ملف الترويسة لصنف الإنارة

- نبدأ بحارس التضمين. الدالة الرئيسية تضمّن ترويسة الإنارة، وتضمّن أيضًا ترويسة الغرفة التي تضمّنّها هي بدورها، فيصل الملف مرتين إلى ملف الترجمة نفسه. الحارس يجعل المرة الثانية تُتجاهل، فلا يُعرّف الصنف مرتين:

```
#ifndef LIGHT_H_
```

- الصفات الثلاث (الرقم والاسم والحالة) في القسم الخاص، والدوال في القسم العام.
- في ملف الترويسة نكتب البادئة الكاملة قبل نوع النص، ولا نفتح فضاء الأسماء أبدًا، لأن الترويسة تُنسخ داخل كل ملف يضمّنّها فينتقل إليه كل ما فيها:

```
std::string
```

- نعلن عن منشئ افتراضي، ومنشئ يأخذ الرقم والاسم، ومُهدّم ظاهري كما يفعل المدرّس، ودالة طباعة تأخذ مرجعًا لتدفق الإخراج وتعيده نفسه، حتى نستطيع الطباعة في الشاشة أو في أي تدفق آخر.

الخطوة الثالثة: ملف التنفيذ لصنف الإنارة

- المنشئ الافتراضي يستخدم قائمة التهيئة: الرقم يساوي -1، والاسم كلمة تعني «لا يوجد»، والحالة «خطأ» أي مطفأة. هكذا نحقق القيمة الافتراضية المكتوبة في المخطط.

- المنشئ الثاني يستخدم قائمة التهيئة أيضًا. في التعبير التالي، الاسم خارج القوسين هو صفة الكائن، والاسم داخل القوسين هو المعامل، لذلك يعمل بشكل صحيح:

```
id(id)
```

- دالة التشغيل تجعل الحالة «صحيح» وتعيد 0، ودالة الإطفاء تجعلها «خطأ» وتعيد 0.
- القاعدة في المشروع كله: 0 يعني نجاح و -1 يعني خطأ. الإنارة الواحدة لا يمكن أن تفشل في التشغيل أو الإطفاء، لذلك تعيد 0 دائمًا.

- دالة الفحص تعيد الحالة كما هي.

- في دوال التعيين للمعامل نفس اسم الصفة، فنفرّق بينهما بالمؤشر إلى الكائن الحالي:

```
this->name = name
```

- فوق كل دالة تعليق بصيغة المدرّس: وصف قصير، ثم كل معامل ونوعه، ثم القيمة المعادة.

الخطوة الرابعة: ملف الترويسة لصنف الغرفة

- تضمّن ترويسة الإنارة، لأن الغرفة تحتوي كائنات إنارة.

- نعرّف ثابتًا للحد الأقصى قيمته 5 بدل تكرار الرقم في كل مكان، ثم نعلن المصفوفة بهذا الحجم:

```
const int MAX_LIGHTS = 5;
```

- الصفات الخمس مطابقة للمخطط تمامًا، وكلها خاصة.
- لكل صفة دالة جلب ودالة تعيين. للمصفوفة دالتان تعملان على خانة واحدة: جلب نسخة من الإنارة في خانة معينة، ووضع إنارة في خانة معينة. وأضفنا دالة مساعدة تضيف إنارة بعد آخر إنارة مستخدمة.

الخطوة الخامسة: ملف التنفيذ لصنف الغرفة

- عند إنشاء كائن الغرفة تستدعي اللغة تلقائيًا المنشئ الافتراضي للإنارة خمس مرات، مرة لكل خانة. لذلك لا نذكر المصفوفة في قائمة التهيئة، وتبدأ كل خانة برقم 1 وحالة مطفأة. أما عدد الإنارات فيبدأ صفرًا.
- المصفوفة جزء من كائن الغرفة، والكائن متغير محلي في الدالة الرئيسية، أي أنه على المكّس. نحن لم نحجز أي ذاكرة من الكومة بأنفسنا، فلا يوجد ما نحذفه ولا يوجد تسريب للذاكرة، ولذلك المهدّم فارغ.
- التحقق من رقم الخانة: الخانات المستخدمة تبدأ من 0 وتنتهي عند عدد الإنارات ناقص واحد. كل دالة تأخذ رقم خانة تفحصه أولاً، فإن كان سالبًا أو أكبر من عدد الإنارات أو مساويًا له أعادت -1 ولم تلمس المصفوفة.
- دالة تعيين العدد لا تقبل إلا القيم من 0 إلى 5، ودالة الإضافة ترفض عندما تمتلئ الغرفة. إذن العدد لا يتجاوز 5 أبدًا، ولهذا يبقى أي رقم خانة مقبول بين 0 و 4، فلا نكتب أبدًا خارج حدود المصفوفة.
- دالة تشغيل إنارة في الغرفة تفحص الخانة ثم تستدعي دالة التشغيل للكائن الموجود فيها وتعيد نتيجتها. دالة الإطفاء تعمل بالطريقة نفسها.
- دالة طباعة الغرفة تطبع الرقم والاسم والموقع والعدد، ثم تمر بحلقة على الإنارات المستخدمة وتستدعي دالة الطباعة لكل كائن إنارة. هذا مثال على كائن يحتوي كائنات أخرى ويتعاون معها.

الخطوة السادسة: الدالة الرئيسية وقراءة ملف البيانات

- أولاً نفحص عدد الوسائط، ويجب أن يكون اثنين: اسم البرنامج واسم الملف. إن لم يكن كذلك نطبع سطر الاستخدام في مجرى الأخطاء ونعيد -1، تمامًا كما في مثال الترتيب الذي أعطاه المدرّس. الطرفية تعرض هذه القيمة على أنها 255.
- نفتح الملف بتدفق إدخال من ملف ونؤكد أنه فُتح فعليًا:

```
std::ifstream
```

- دالة القراءة تقرأ الملف بعد كلمة بعامل الاستخراج، وبالترتيب: الكلمة المفتاحية الأولى، ثم رقم الغرفة واسمها وموقعها، ثم الكلمة المفتاحية الثانية وعدد الإنارات، ثم سطر لكل إنارة فيه رقمها واسمها، ثم كلمة النهاية. الكلمات المفتاحية الثلاث هي:

```
Room / Light / End
```

- بعد كل قراءة نفحص هل فشل التدفق، ونقارن الكلمة المفتاحية بالمتوقع. عند أي خطأ نكتب رسالة واضحة في مجرى الأخطاء ونعيد -1.

- لكل إنارة ننشئ كائنًا بالرقم والاسم ثم نضيفه إلى الغرفة عن طريق دالة الإضافة. إذا ذكر الملف سبع إنارات نقرأ السبع كلها حتى يبقى موضع القراءة صحيحًا ونصل إلى كلمة النهاية، ونحتفظ بأول خمس، ونطبع تحذيرًا لكل إنارة زائدة.
- إذا انتهى الملف قبل الألوان تفشل القراءة، فنطبع خطأ ويخرج البرنامج بهدوء دون انهيار. وإذا لم نجد كلمة النهاية في مكانها نطبع رسالة تقترح فحص عدد الإنارات، لأن هذا هو السبب المعتاد.
- يجب أن يكون كل اسم كلمة واحدة، لأن عامل الاستخراج يتوقف عند أول مسافة.

الخطوة السابعة: المخرجات الثلاث المطلوبة

- أولاً نطبع الغرفة بعد التحميل مباشرة، فتظهر كل الإنارات مطفاة لأن هذه هي القيمة الافتراضية.
- ثانيًا حلقة من الخانة 0 إلى آخر خانة مستخدمة تشغل كل إنارة عن طريق دالة الغرفة، ثم نطبع الغرفة فتظهر كلها مضاءة.
- ثالثًا حلقة مماثلة تطفئ كل الإنارات، ثم نطبع الغرفة.
- رقم الخانة المطبوع بجانب كل إنارة هو ما نمرره لدالة التشغيل، وهو يختلف عن رقم الإنارة المكتوب في الملف.
- نص الواجب في الخطوة الثالثة يذكر أيضًا إغلاق الستائر، لكن المخطط في هذه النسخة لا يحتوي أي صنف للستائر، ويبدو أن الجملة من نسخة لاحقة من الواجب. لذلك لم نخترع صنفًا جديدًا، واكتفينا بإطفاء الإنارات، وكتبنا تعليقًا في الكود وملاحظة في ملف الشرح.

الخطوة الثامنة: البناء والاختبار

- ملف البناء يصنع البرنامج التنفيذي، وأمر الترجمة نفسه يعمل مع المترجمين كليهما دون أي تحذير مع خيارات التحذير الصارمة.
- جربنا حالات كثيرة: التشغيل بلا ملف، وملف غير موجود، وملف فيه سبع إنارات، وملف مقطوع في المنتصف، وعدد إنارات لا يطابق الأسطر، وكلمة مفتاحية خاطئة. كل حالة عولجت برسالة واضحة دون انهيار.
- أداة فحص الذاكرة أظهرت صفر أخطاء وصفر تسريب:

valgrind